Advancing Human-Robot Interaction with Real-Time Gesture Commands

Master's Thesis

Bennit Strack



August 2026
First Advisor: Prof. Dr. Sören Pirk
Second Advisor: M.Sc. Samira Huber

Christian-Albrechts-Universität zu Kiel
Department of Computer Science
Visual Computing and Artificial Intelligence
Neufeldtstr. 6, 24098 Kiel
Germany

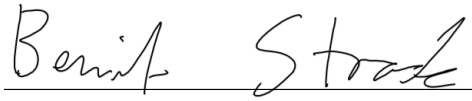
Declaration

I hereby declare that I have prepared this thesis independently and without outside help and have not used any sources or aids other than those specified.

The submitted written version of the thesis is identical to the digital version.

Furthermore, I assure you that this thesis has not yet been submitted as a thesis elsewhere.

Kiel, 21 August 2026



Bennit Strack

Abstract

Traditional robotic teleoperation in industrial environments relies heavily on physical controllers, such as joysticks and tablets, which occupy operators' hands and increase cognitive load. To address these ergonomic limitations, this thesis presents the development and evaluation of a robust, hands-free vision-based gesture control system deployed entirely on edge computing hardware. The proposed solution implements a decoupled hybrid architecture that separates spatial feature extraction from temporal classification. Utilizing the Google MediaPipe framework, precise three-dimensional hand landmarks are extracted from raw video feeds. These spatial coordinates are subsequently ingested by a computationally lightweight Gated Recurrent Unit for real-time sequence classification. This pipeline was successfully integrated via the Robot Operating System to control a Boston Dynamics Spot quadruped, effectively bridging the gap between advanced computer vision and practical robotic teleoperation.

Empirical validation of the system demonstrated exceptional quantitative performance, achieving a validation accuracy of 91.63% on the large-scale Jester dataset while maintaining sub-millisecond inference latencies on the NVIDIA Jetson AGX Orin. Beyond computational metrics, a comprehensive two-part user study provided critical insights into human-robot interaction preferences. When evaluating fundamental interaction types, the results definitively established that operators strongly prefer mechanically straightforward, static gestures over temporally complex or multi-stage movements. Static interactions fundamentally minimized cognitive load and allowed for effortless, intuitive error recovery, proving highly advantageous in dynamic industrial settings.

Furthermore, during a simulated warehouse manipulation task, participants favored the vision-based gesture interface over standard physical tablet controllers. The ability to maintain completely free hands during operation emerged as the decisive pragmatic advantage, confirming that the elimination of physical hardware constraints significantly enhances the subjective user experience. Ultimately, this work demonstrates that vision-based spatial-temporal gesture recognition represents a fundamental, human-centric shift in the future of robotics interaction, laying a critical foundation for highly intuitive and scalable industrial teleoperation.

Zusammenfassung

Die traditionelle robotergestützte Teleoperation in industriellen Umgebungen stützt sich stark auf physische Controller wie Joysticks und Tablets, welche die Hände der Bediener binden und die kognitive Belastung erhöhen. Um diese ergonomischen Einschränkungen zu adressieren, präsentiert diese Arbeit die Entwicklung und Evaluierung eines robusten, freihändigen und visionsbasierten Gestensteuerungssystems, das vollständig auf Edge-Computing-Hardware implementiert wurde. Die vorgeschlagene Lösung nutzt eine entkoppelte, hybride Architektur, welche die räumliche Merkmalsextraktion von der zeitlichen Klassifizierung trennt. Unter Verwendung des Google MediaPipe-Frameworks werden präzise dreidimensionale Hand-Landmarks aus rohen Videostreams extrahiert. Diese räumlichen Koordinaten werden anschließend von einer rechenleichten Gated Recurrent Unit für die Echtzeit-Sequenzklassifizierung verarbeitet. Diese Pipeline wurde erfolgreich über das Robot Operating System integriert, um einen Boston Dynamics Spot Quadrupeden zu steuern, wodurch die Lücke zwischen fortschrittlicher Computer Vision und praktischer robotergestützter Teleoperation effektiv geschlossen wird.

Die empirische Validierung des Systems demonstrierte eine herausragende quantitative Leistung, wobei eine Validierungsgenauigkeit von 91,63 % auf dem umfangreichen Jester-Datensatz erreicht wurde, während gleichzeitig Sub-Millisekunden-Inferenzlatenzen auf dem NVIDIA Jetson AGX Orin aufrechterhalten wurden. Über die rechnerischen Metriken hinaus lieferte eine umfassende zweiteilige Nutzerstudie kritische Einblicke in die Präferenzen der Mensch-Roboter-Interaktion. Bei der Evaluierung grundlegender Interaktionstypen zeigten die Ergebnisse eindeutig, dass Bediener mechanisch einfache, statische Gesten gegenüber zeitlich komplexen oder mehrstufigen Bewegungen stark bevorzugen. Statische Interaktionen minimierten die kognitive Belastung grundlegend und ermöglichten eine mühelose, intuitive Fehlerbehebung, was sich in dynamischen industriellen Umgebungen als äußerst vorteilhaft erweist.

Darüber hinaus bevorzugten die Teilnehmer während einer simulierten Lager-Manipulationsaufgabe die visionsbasierte Gestenschnittstelle gegenüber standardmäßigen physischen Tablet-Controllern. Die Möglichkeit, die Hände während des Betriebs vollständig frei zu haben, kristallisierte sich als entscheidender pragmatischer Vorteil heraus, was bestätigt, dass die Eliminierung physischer Hardwarebeschränkungen die subjektive Nutzererfahrung erheblich verbessert. Letztendlich demonstriert diese Arbeit, dass die visionsbasierte räumlich-zeitliche Gestenerkennung einen grundlegenden, menschenzentrierten Wandel in der Zukunft der Roboterinteraktion darstellt und ein kritisches Fundament für eine hochintuitive und skalierbare industrielle Teleoperation legt.

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Research Objective . . . 2
- 1.3 Approach . . . 3

2 Related Work **5**
- 2.1 Vision-Based Gesture Recognition . . . 5
- 2.2 Efficient Inference and Edge AI . . . 7
  - 2.2.1 Decoupled Spatial-Temporal Architecture . . . 7
  - 2.2.2 Inference Frameworks and Quantization . . . 8
- 2.3 HRI with Quadruped Robots . . . 9
- 2.4 Summary . . . 10

3 Methodology **11**
- 3.1 Rationale for RGB Video-Based Classification . . . 11
- 3.2 Dataset . . . 12
- 3.3 Dataset Preprocessing and Landmark Imputation . . . 13
- 3.4 Decoupled System Architecture . . . 13
- 3.5 Spatial Feature Extraction via MediaPipe . . . 14
- 3.6 Temporal Classification via GRUs . . . 17
  - 3.6.1 Skeletal Augmentation . . . 17
  - 3.6.2 Bidirectional GRU Architecture . . . 20
- 3.7 Hybrid Architecture: Static Gestures and Complex Interactions . . . 21
  - 3.7.1 Static Gesture Classifier . . . 21
  - 3.7.2 Two-Hand Gestures . . . 23
  - 3.7.3 Continuous Parameter Control . . . 23
  - 3.7.4 Stacked Gestures . . . 24
- 3.8 Pipeline Filters and Conflict Resolution . . . 24
  - 3.8.1 Temporal Stability Filter . . . 24
  - 3.8.2 Priority Filter . . . 25
- 3.9 Gesture-to-Action Mapping . . . 25
  - 3.9.1 Pre-Filtering . . . 25
  - 3.9.2 Safety Architecture . . . 26
- 3.10 Baseline Architectures and Benchmarking . . . 26
  - 3.10.1 Spatial Extraction Baseline . . . 26
  - 3.10.2 Temporal Skeletal Baselines . . . 27
  - 3.10.3 End-to-End Processing Baseline . . . 28
- 3.11 Summary . . . 28

4 Implementation & Experiments **29**
- 4.1 Hardware Setup . . . 29
- 4.2 Training Infrastructure & Dataset Implementation . . . 30
- 4.3 Spatial Feature Extraction . . . 31

4.4 Temporal Classification Models . . . 31
4.5 Pipeline Filters & Conflict Resolution . . . 33
4.5.1 Stability Filter . . . 33
4.5.2 Priority Filter . . . 34
4.6 Real-Time Inference Architecture & Optimizations . . . 34
4.7 Robot Control API Integration . . . 36
4.8 User Interface . . . 36
4.9 Deployment & System Orchestration . . . 39
4.10 User Study Design . . . 40
4.10.1 Experimental Setup & Task Design . . . 40
4.10.2 Survey Methodology & Questionnaire Rationale . . . 41
4.11 Summary . . . 42

5 Results **43**
5.1 Spatial Feature Extraction Benchmark . . . 43
5.1.1 YOLO vs MediaPipe Detection Rates . . . 43
5.1.2 MediaPipe Landmark Detection Analysis . . . 45
5.2 Temporal Classification . . . 48
5.2.1 Impact of Landmark Interpolation . . . 53
5.2.2 Comparison with State-of-the-Art . . . 54
5.3 Hardware Optimization & System Performance . . . 56
5.3.1 Model Inference Latency . . . 56
5.3.2 Energy Efficiency and Resource Utilization . . . 57
5.4 User Study Results . . . 59
5.4.1 Core Gesture Mechanics Evaluation . . . 59
5.4.2 System Mechanics & Tablet Comparison . . . 61
5.4.3 Alignment with Broader Academic Consensus . . . 62
5.4.4 Overall Gesture Preference . . . 63
5.5 Summary . . . 64

6 Limitations **65**
6.1 Algorithmic and Architectural Constraints . . . 65
6.2 Hardware and Embodiment Constraints . . . 66
6.3 Dataset and Generalization Biases . . . 67

7 Future Work **69**
7.1 Continuous Streaming Architectures . . . 69
7.2 True Bimanual Interaction . . . 69
7.3 Online Few-Shot Learning and Continual Adaptation . . . 70
7.4 Resolving Morphological Domain Shift . . . 70
7.5 Hardware Feedback and Modality Evaluation . . . 71

8 Conclusion **73**

List of Figures **75**

List of Tables **77**

Bibliography **79**

A Appendix **85**
A.1 Full Confusion Matrix 85

Notation

Abbreviations

AdamW	adaptive moment estimation with decoupled weight decay
AI	artificial intelligence
API	application programming interface
ARM	Advanced RISC Machines
CNN	convolutional neural network
CPU	central processing unit
CUDA	Compute Unified Device Architecture
FLOPs	floating-point operations
FPS	frames per second
GB	gigabyte
GFLOPs	giga floating-point operations
GPU	graphics processing unit
GRU	gated recurrent unit
HRI	human-robot interaction
HTML	HyperText Markup Language
IP	Internet Protocol
LED	light-emitting diode
LSTM	Long Short-Term Memory
MAC	multiply-accumulate operation
MFLOPs	mega floating-point operations
MHz	megahertz
ONNX	Open Neural Network Exchange
RGB	red, green, blue
RISC	reduced instruction set computer
ROI	region of interest
SDK	software development kit
SLAM	simultaneous localization and mapping
ST-GCN	spatial temporal graph convolutional network
TSM	Temporal Shift Module
UI	user interface
USB	Universal Serial Bus
VR	virtual reality
YOLO	You Only Look Once

1 Introduction

The deployment of autonomous and semi-autonomous mobile robots is rapidly expanding beyond highly controlled industrial cages into dynamic, human-centric environments. As robots increasingly work alongside human operators in shared spaces, the methods by which humans command and interact with these machines must evolve. Traditional hardware interfaces, while functional, often fail to meet the ergonomic and cognitive demands of complex, unstructured tasks. Developing natural, intuitive, and computationally efficient human-robot interaction (HRI) modalities is therefore a critical frontier in modern robotics research.
This chapter establishes the foundation for this thesis. Section 1.1 outlines the operational and technical challenges that motivate the shift toward vision-based gesture control on edge devices. Section 1.2 defines the core research questions guiding this work, and Section 1.3 summarizes the methodological approach while providing a structural overview of the subsequent chapters.

1.1 Motivation

Agile mobile robots, such as Boston Dynamics Spot, are increasingly deployed in unstructured, real-world environments ranging from industrial inspections to search-and-rescue operations [4]. As these systems integrate into daily workflows, the efficiency of HRI becomes a critical bottleneck. Currently, operators rely heavily on physical controllers, such as joysticks or tablet interfaces, to maneuver the robot and trigger specific routines. While functional, these manual interfaces impose substantial limitations on the operator. They bind the user's hands, demand constant visual attention, and induce high cognitive load [1]. In complex industrial scenarios where operators must maintain situational awareness or manipulate tools, relying on a physical device is highly impractical. This issue is further exacerbated when an operator's hands become soiled by machinery grease or dirt, rendering touch-based tablet controllers impractical without prior cleaning.
Vision-based gesture control offers a compelling, natural alternative to traditional hardware controllers. By translating intuitive hand movements directly into robotic actions, gesture interfaces eliminate the need for intermediary devices. This allows the operator to maintain completely free hands and sustained visual focus on the robot and the surrounding environment. Consequently, the operational workflow

becomes significantly more seamless, enabling workers to command complex robotic systems effortlessly while engaging in their primary physical tasks.
Despite the inherent ergonomic advantages of gesture control, deploying state-of-the-art computer vision models onto mobile robotic platforms presents a profound technical challenge. Modern gesture recognition architectures predominantly rely on large-scale, end-to-end 3D convolutional neural networks (3D-CNNs) or computationally demanding Transformer models [16, 69]. While these models achieve exceptional classification accuracy, they are exceedingly computationally expensive. Autonomous mobile robots operate on strictly limited energy budgets and rely on lightweight edge hardware, such as the NVIDIA Jetson series, rather than high-performance desktop graphics processing units (GPUs). Attempting to run conventional 3D-CNNs natively on such edge devices either drains the robot's battery excessively rapidly or fails entirely to meet the strict sub-100-millisecond real-time latency requirements necessary for safe, responsive teleoperation [30]. Furthermore, offloading the computational burden to external cloud servers introduces unpredictable network latency and exposes the system to connection drops, which is unacceptable for mission-critical robotic control.
Consequently, there exists a substantial gap between high-precision artificial intelligence (AI) research and practical, resource-constrained robotic deployment. The robotics industry urgently requires a highly efficient, lightweight gesture recognition architecture that bridges this divide, delivering robust real-time performance without consuming the edge hardware's compute budget or compromising the robot's operational autonomy.

1.2 Research Objective

The primary objective of this thesis is to design, implement, and evaluate a highly efficient, real-time vision-based gesture control system for the Boston Dynamics Spot robot, optimized specifically for resource-constrained edge computing hardware.
To systematically address this overarching goal, the research is structured around three core research questions (RQs):

- **RQ1 (Architecture & AI):** How can a decoupled gesture recognition pipeline, separating spatial landmark extraction from temporal classification, be designed to maximize classification accuracy while significantly minimizing computational complexity compared to state-of-the-art end-to-end computer vision models?

- **RQ2 (Hardware & Efficiency):** To what extent can hardware-specific optimizations, such as TensorRT engine compilation, reduce the end-to-end inference latency and power consumption when the gesture pipeline is deployed natively on an edge platform such as the NVIDIA Jetson AGX Orin?

- **RQ3 (Usability & HRI):** How do users evaluate the usability of the developed gesture control system, specifically regarding their ergonomic preferences

among the five distinct gesture topologies, and the system's operational advantages compared to a traditional tablet controller during a simulated industrial manipulation task?

This thesis focuses strictly on red, green, blue (RGB) video-based skeletal tracking and predefined gesture catalogs, deliberately excluding depth-sensor fusion or open-vocabulary intention recognition. By doing so, it isolates and evaluates the algorithmic efficiency of the proposed hybrid classification architecture, providing a definitive benchmark for deploying robust human-robot interfaces on highly constrained edge devices.

1.3 Approach

To address the outlined research objectives without introducing redundancy, this thesis systematically builds the proposed gesture control system across the following chapters. First, **Chapter 2** reviews the related work and state-of-the-art developments in robotic teleoperation, computer vision, and edge computing, establishing the theoretical foundation for this research.
Building upon these fundamentals, **Chapter 3** introduces the core methodology. It details the theoretical design of a highly decoupled, hybrid classification architecture. By utilizing Google MediaPipe for spatial landmark extraction and a lightweight gated recurrent unit (GRU) for temporal classification, the architecture explicitly avoids the computational bottlenecks of traditional end-to-end 3D-CNNs. This chapter also defines the logic for complex gesture interactions, including continuous parameter sliders and multi-stage stacked gestures, alongside the necessary safety and priority filters.
Chapter 4 documents the practical implementation of this architecture onto the Boston Dynamics Spot's NVIDIA Jetson AGX Orin edge platform. It covers the asynchronous multi-threading infrastructure, the compilation of PyTorch models into highly optimized TensorRT engines to maximize inference speed, and the orchestration of the system via Docker. Furthermore, it outlines the exact experimental setup for the hardware benchmarks and the design of the qualitative user study.
The comprehensive evaluation of the system is presented in **Chapter 5**. This chapter analyzes the empirical results, benchmarking the proposed architecture's classification accuracy, computational floating-point operations (FLOPs), and parameter efficiency against heavier baseline models. It comprehensively profiles the real-time latency and power consumption on the edge hardware, and combines the statistical findings from the user study to determine ergonomic preferences.
Finally, **Chapter 6** critically discusses the inherent limitations of the developed system, while **Chapter 7** proposes specific avenues for future work. **Chapter 8** concludes the thesis by summarizing the primary achievements and explicitly answering the three research questions.

2
Related Work

Developing a robust, real-time gesture control interface for a mobile quadruped robot requires synthesizing advancements from multiple distinct disciplines. This chapter contextualizes the proposed architecture by analyzing the current state-of-the-art across three fundamental domains. First, Section 2.1 evaluates vision-based gesture recognition methodologies, detailing the computational trade-offs between monolithic deep learning architectures and decoupled skeletal representations. Second, Section 2.2 investigates efficient inference frameworks tailored for resource-constrained edge computing environments. Finally, Section 2.3 examines the ergonomic and environmental challenges of HRI in field deployments. By critically evaluating these existing paradigms, this chapter identifies the specific methodological gaps and hardware bottlenecks that the subsequent thesis architecture aims to resolve.

2.1 Vision-Based Gesture Recognition

The field of vision-based gesture recognition has historically been dominated by end-to-end deep learning architectures that process raw RGB video volumes directly. Early breakthroughs relied heavily on dense 3D-CNNs to extract intertwined spatial and temporal features simultaneously. While these architectures achieve remarkable classification accuracy on constrained academic datasets, their computational complexity scales cubically with the spatio-temporal resolution of the input video. This is due to the volumetric nature of the convolution operation, which mandates the processing of every pixel across the height, width, and temporal depth of the input tensor. For example, the deformable ResNeXt3D architecture, despite its optimizations, still demands nearly 59.76 giga floating-point operations (GFLOPs) for a single inference pass [22]. When deployed on resource-constrained platforms, such intensive operations enforce severe limitations. As demonstrated by Rehman et al. [51], researchers deploying 3D-CNN models on edge devices are often forced to artificially restrict their classification vocabulary from a standard 27-class dataset down to merely 15 gestures to prevent the hardware from exhausting its memory and processing time limits.

More recently, the paradigm has shifted toward Video Vision Transformers (ViTs), which leverage self-attention mechanisms to capture long-range temporal dependencies across video frames. Architectures such as ViViT [2] and the Video Swin Trans-

former [34] have established new state-of-the-art benchmarks in action recognition. However, the global attention mechanism inherently exhibits quadratic computational complexity $\mathcal{O}(N^2)$ with respect to the sequence length N. As the temporal duration of a gesture increases, the number of attention calculations between frame tokens grows quadratically. Consequently, these transformer-based models require substantial parameter counts and considerable memory bandwidth, rendering them unsuitable for deployment on battery-powered edge robotics platforms where power consumption dictates operational endurance.

Furthermore, the robustness of these end-to-end monolithic architectures is heavily dependent on the sheer volume of their training data. Models trained on raw pixels are highly susceptible to overfitting on environmental artifacts, such as lighting variations or static background textures, rather than learning the underlying kinematic motion of the gesture [10]. This vulnerability necessitates the curation of extensive, crowdsourced video datasets, such as the 20BN-Jester dataset [41], which provides the requisite environmental variance to ensure baseline generalization.

To mitigate the computational bottleneck of raw pixel processing, contemporary research has pivoted toward skeleton-based gesture recognition. By abstracting the human operator into a sparse set of anatomical landmarks, the dimensionality of the input space is drastically reduced before any temporal classification occurs. Spatial temporal graph convolutional networks (ST-GCNs) [66] pioneered this approach by modeling the human skeleton as a dynamic graph, where physical joints serve as nodes and bones act as edges. While ST-GCNs significantly reduce the FLOP footprint compared to 3D-CNNs, they still perform computationally expensive graph convolutions across the entire temporal sequence. Subsequent iterations, such as PoseConv3D [14], attempted to bridge the gap by converting 2D skeletal data into 3D heatmap volumes, effectively trading spatial precision for enhanced temporal robustness, though at the cost of reintroducing volumetric convolution overhead.

However, transitioning from raw pixels to sparse skeletal representations introduces distinct challenges. In a comprehensive representation-centric survey, Liu et al. [33] demonstrate that raw absolute coordinate vectors are highly susceptible to global translation effects. Consequently, models trained directly on normalized pixel coordinates are prone to overfitting on macro motion and absolute spatial positioning, rather than learning the underlying relative kinematics. In the context of mobile robotics, this creates a critical spatial bias that fails when the camera undergoes continuous physical translation. As suggested by Liu et al., overcoming this bias requires transitioning to local coordinate representations and employing extensive viewpoint augmentation strategies to decouple the kinematic essence of the gesture from the absolute camera frame.

The realization that spatial feature extraction and temporal classification should be strictly decoupled has driven the most recent advancements in efficient gesture recognition. Early precursors to this decoupled architecture, such as the hybrid approach proposed by Cao et al. [5], utilized a 3D-CNN solely for spatial feature extraction, passing the condensed representations to a Long Short-Term Memory (LSTM) network for temporal modeling. While this represented a crucial architectural evolution, the reliance on a 3D-CNN for the initial spatial extraction remained a critical

performance bottleneck. True edge efficiency requires completely decoupling the temporal sequence modeling from dense pixel convolutions, a transition that forms the foundational architecture of the proposed robotic teleoperation pipeline.

2.2 Efficient Inference and Edge AI

Deploying vision-based recognition models on autonomous robotic platforms introduces severe hardware constraints that are fundamentally absent in cloud-based server environments. Mobile quadrupeds, such as the Boston Dynamics Spot, must allocate the majority of their onboard power and computational budget to core locomotion, balance, and Simultaneous localization and mapping (SLAM) processes. Consequently, the remaining payload capacity for auxiliary perception models is extremely limited. While 2D convolutional approaches such as the Temporal Shift Module (TSM) [30] achieve significant efficiency gains over volumetric 3D-CNNs by shifting feature maps across the temporal dimension, they still process raw video frames and require several GFLOPs per inference pass, exceeding the strict energy constraints of lightweight edge devices.

2.2.1 Decoupled Spatial-Temporal Architecture

To achieve true real-time performance on constrained hardware, the computational burden of processing background pixels must be entirely eliminated. Frameworks such as MediaPipe [67] provide a highly optimized solution by deploying a cascaded Region of interest (ROI) extraction pipeline. This pipeline operates in two distinct phases: a lightweight palm detector rapidly scans the full image to produce a bounded crop, which is then fed into a highly localized 3D landmark regressor. By regressing 21 sparse 3D skeletal landmarks, the input dimensionality is compressed from a dense RGB matrix into a minimal coordinate vector. This spatial abstraction reduces the FLOP footprint by several orders of magnitude compared to global object detectors.

Once the spatial features are extracted, the temporal sequence must be modeled efficiently. Recurrent Neural Networks, specifically LSTM networks and GRUs [9], excel at capturing sequential dependencies in these low-dimensional vectors. The GRU architecture is particularly advantageous for edge inference because it merges the internal cell state with the hidden state and utilizes only two gating mechanisms (reset and update). This structural simplification achieves comparable predictive performance to LSTMs on shorter temporal sequences while requiring roughly 25% fewer mathematical parameters, translating directly to lower memory allocations and reduced active power consumption [7, 52].

However, recurrent networks trained on structured coordinate data are highly susceptible to lazy learning, where the model simply memorizes absolute coordinate paths rather than the underlying kinematic motion. To combat this, advanced regularization techniques are required. Loshchilov and Hutter [37] demonstrated that standard L2 regularization is mathematically flawed when combined with adaptive

gradient algorithms. Their proposed AdamW optimizer mathematically decouples the weight decay from the gradient update, drastically improving generalization on highly correlated spatial sequences.
The combination of MediaPipe skeletal extraction and recurrent temporal modeling represents the current state-of-the-art for lightweight gesture recognition. A direct parallel to the architecture proposed in this thesis is the system developed by Sabbarese et al. [52], who similarly employ a pipeline consisting of MediaPipe coupled with a GRU classifier. They supplement their training with an engine for generating synthetic video data to augment the lack of real-world training instances, successfully evaluating their system across the entire 27-class Jester dataset with an accuracy of 88.65%. However, a critical vulnerability in their methodology is the direct utilization of raw, normalized screen coordinates as input features. While raw screen coordinates perform adequately in static extended reality environments, such as an operator sitting directly in front of a stationary webcam, they are inherently brittle when deployed on a moving robotic platform. As a quadruped robot traverses uneven terrain, the camera experiences continuous translation and rotation, causing the operator's hands to shift erratically across the image plane. A model trained on absolute spatial positions will fail entirely when presented with these translational shifts. Overcoming this requires the classifier to be explicitly trained to achieve translational and scale invariance through extensive spatial augmentation.

2.2.2 Inference Frameworks and Quantization

Beyond the topological design of the neural network, the choice of the runtime environment heavily dictates the final inference latency on edge hardware. Recent studies in edge AI, such as the lightweight hybrid architecture developed by Peng et al. [49], demonstrate that embedded temporal skeletal modeling on platforms such as the NVIDIA Jetson Nano typically achieves response latencies just under 300 milliseconds. While sufficient for basic interaction, seamless robotic teleoperation demands significantly faster execution.
Achieving ultra-low latency requires optimizing the computational graph directly for the target hardware. Mukherjee and Cha [45] provide a comprehensive analysis of GPU-accelerated bottlenecks, detailing how extensive post-training optimizations can bypass the overhead of native Python execution frameworks. By utilizing compilers such as NVIDIA TensorRT or the Open Neural Network Exchange (ONNX) Runtime, models can undergo profound structural optimizations. Techniques such as operator fusion, which merges sequential matrix multiplications and activation functions into a single executable kernel, drastically reduce memory access latency. Furthermore, quantizing the model weights from 32-bit floating-point (FP32) down to 16-bit floating-point (FP16) halves the memory bandwidth requirement while maintaining classification fidelity. These compiler-level optimizations are essential for pushing the gesture recognition pipeline from a standard conceptual prototype into a truly real-time, field-ready teleoperation system.

2.3 HRI with Quadruped Robots

The physical methodology by which human operators interface with complex mobile robots, particularly quadruped platforms, directly impacts the efficiency and cognitive load of teleoperation. Traditionally, industrial and exploratory robots are controlled via tethered teaching pendants, handheld joysticks, or tablet-based graphical user interfaces (UIs). While these physical controllers offer deterministic actuation and high perceived precision, they impose substantial ergonomic constraints on the operator. As evaluated by Ahmad et al. [1], traditional interfaces require the operator to constantly divide their visual attention between the screen and the physical robot navigating the environment. This visual context switching drastically increases the operator's cognitive load, particularly in hazardous or rapidly changing field environments.

To alleviate this burden, research in HRI has increasingly focused on hands-free, touchless modalities. The empirical study conducted by Kristof et al. [28] comprehensively compared traditional teaching pendants against gesture-based control interfaces. Their findings revealed a compelling user preference: despite the tablet interfaces offering marginally higher objective positioning precision, a substantial 85% of participants preferred the gesture control system. The users reported that the gesture interface felt significantly more intuitive and physically liberating, underscoring the subjective value of natural kinematic interaction over rigid mechanical controls.

However, implementing these spatial gesture controls in real-world robotics introduces severe environmental constraints. Early attempts utilized active infrared depth sensors to capture volumetric hand data [53]. While highly accurate indoors, these active sensors frequently fail in outdoor environments where ambient solar radiation saturates the infrared receivers, rendering the robot blind to the operator's commands [15]. To bypass the reliance on fragile optical sensors entirely, some researchers have pivoted to immersive virtual reality (VR) teleoperation, such as the framework proposed by Lipton et al. [31]. However, deploying such immersive systems in field robotics introduces profound ergonomic and safety limitations. VR headsets completely isolate the operator's vision from their immediate physical surroundings, which is a critical risk during active field deployments where the operator must remain acutely aware of the moving quadruped and environmental hazards. Furthermore, requiring a dedicated headset prohibits spontaneous interaction, as human co-workers cannot naturally assume control of the robot without first donning specialized hardware.

Conversely, pure RGB vision-based gesture recognition offers exceptional ergonomic flexibility without compromising safety. Monocular vision systems, such as the DroNet architecture developed by Loquercio et al. [35] for autonomous unmanned aerial vehicle navigation, demonstrate that standard RGB cameras can successfully replace heavy, active LiDAR or depth payloads if the underlying neural network is

sufficiently robust. As supported by Vanc et al. [61], RGB gesture control requires zero additional hardware on the operator and maintains full situational awareness. Despite the clear ergonomic superiority of vision-based gesture control, a significant research gap persists in its deployment on quadruped robots. Because highly accurate gesture recognition historically relied on the computationally intensive architectures discussed in Section 2.1, existing quadruped control systems are often forced to offload the visual processing to external edge servers or cloud infrastructure. This reliance on external processing introduces latency and creates a single point of failure. As demonstrated by Staniaszek et al. [57] in their long-term autonomous inspection study, industrial environments frequently suffer from severe Wi-Fi degradation, packet loss, and localized network blackouts. A gesture control system dependent on wireless offloading will inevitably fail in these environments. Therefore, a critical need exists for a gesture control system that combines the ergonomic benefits of touchless interaction with a spatio-temporal architecture lightweight enough to run entirely natively on the robot's onboard payload computer. By processing the RGB video stream locally on the edge hardware, the system eliminates network dependency, ensuring deterministic, ultra-low latency responses regardless of environmental connectivity. Addressing this precise intersection of algorithmic efficiency, environmental robustness, and HRI flexibility is the primary objective of this thesis.

2.4 Summary

In summary, this chapter reviewed the state-of-the-art in vision-based gesture recognition, edge AI acceleration, and HRI. The analysis demonstrated that while monolithic deep learning models provide high accuracy, they are too computationally expensive for resource-constrained edge devices. Conversely, existing lightweight methods often sacrifice the spatio-temporal nuance required for complex robotic teleoperation. Therefore, a critical need exists for an architecture that decouples spatial tracking from temporal classification to ensure real-time, low-latency inference.

3 Methodology

This chapter details the theoretical and practical methodology underpinning the proposed vision-based robotic gesture control system. First, the rationale for utilizing RGB video-based classification over traditional depth-sensing hardware is established, followed by an overview of the dataset strategy utilized to train the temporal models. The core of the chapter outlines the system's decoupled software architecture, detailing the spatial feature extraction pipeline powered by MediaPipe and the subsequent temporal classification driven by GRUs. Furthermore, the implementation of a hybrid architectural design to accommodate complex stacked and slider-based interactions is discussed, alongside the pipeline filtering mechanisms designed to ensure stable control outputs. Finally, the chapter defines the semantic gesture-to-action mappings used to teleoperate the Spot robot and establishes the scientific benchmarking rationale used to objectively evaluate the system against alternative baseline architectures.

3.1 Rationale for RGB Video-Based Classification

Traditional robotic systems often rely on active sensors such as LiDAR or specialized RGB-D (Depth) cameras to perceive 3D environments. While highly accurate, these sensors impose severe limitations on agile edge devices. LiDAR systems are notoriously power-intensive and bulky, drastically increasing the Size, Weight, and Power (SWaP) footprint of the robotic platform. Furthermore, structured-light and time-of-flight depth cameras frequently fail outdoors due to interference from ambient sunlight, rendering them highly inconsistent for real-world deployments [35].
To overcome these physical and environmental constraints, this architecture relies exclusively on standard 2D RGB video streams. An RGB-only approach provides a significantly more flexible use case, allowing the system to operate on ubiquitous, low-cost camera hardware in both indoor and outdoor environments. While a single RGB camera lacks native depth perception, modern neural network architectures are now capable of reliably extracting implicit 3D spatial information directly from a 2D image stream [40]. This shift from hardware-based depth to software-inferred depth ensures that the robotic platform remains lightweight and energy-efficient without sacrificing spatial awareness.

3.2 Dataset

The foundational requirement for training the temporal classification model is a robust, large-scale dataset of dynamic hand gestures. Given the extensive data requirements of modern neural networks, curating and annotating a custom dataset of sufficient size for this research was deemed unfeasible. Fortunately, the computer vision community provides a variety of publicly available gesture datasets. Selecting the appropriate dataset requires evaluating several data quality dimensions, primarily completeness, accuracy, and unbiasedness [17]. For training the temporal classifier, unbiasedness is particularly critical. The dataset must feature a high variance of human morphology, execution speeds, and kinematic styles to prevent the model from overfitting to a specific operator's movements.

When evaluating available options, several prominent datasets were considered but ultimately rejected due to a lack of scale and variance. For example, the Spatio-temporal Kinect Interaction Gestures dataset [32] contains only 1,080 videos from six actors, while nvGestures [43] provides 1,532 videos from 20 actors. Even larger datasets such as EgoGesture [68], which features 24,161 videos, are limited by a small pool of only 50 actors. These datasets lack the human morphological variance required to train a robust, real-world inference pipeline.

To satisfy these quality characteristics, the primary architecture of this system is designed around the 20BN-Jester dataset [41]. Comprising 148,092 short video clips across 27 distinct gesture classes, it represents a comprehensive collection of dynamic human hand motions. A critical factor in its selection is its crowdsourced acquisition methodology. Jester was recorded by 1,376 unique individuals performing gestures in unconstrained environments. This unprecedented variance in human morphology and execution styles perfectly aligns with the unbiasedness requirements for training temporal classification models that generalize to diverse human operators. Furthermore, while the dataset's raw videos contain immense environmental variance, such as lighting and background clutter, the proposed decoupled pipeline (detailed in Section 3.4) isolates the temporal classifier from this visual noise.

The immense volume of the Jester dataset is equally critical for achieving high inference accuracy. The original authors demonstrated this dependency by evaluating a baseline 3D-CNN across artificially restricted subsets of the training data [41]. As detailed in Table 3.1, restricting the dataset to 100 videos per class yielded a top-1 accuracy of only 62.40%. Increasing the data volume to the full dataset average of 4,391 videos per class improved the accuracy to 93.87%. This severe performance degradation under constrained data volumes highlights the necessity of utilizing a large-scale dataset to properly train robust spatio-temporal representations.

Table 3.1: Impact of training dataset volume on baseline 3D-CNN top-1 accuracy, as reported by the original Jester dataset authors [41].

Videos per Class	Accuracy [Top-1 %]
100	62.40
200	71.60
500	77.70
1000	85.50
2000	88.30
3000	89.50
4391	93.87

3.3 Dataset Preprocessing and Landmark Imputation

A critical preprocessing step when utilizing decoupled architectures is managing transient spatial tracking failures. As detailed in Chapter 5, the upstream MediaPipe spatial extractor frequently drops tracking for brief periods (typically 1–4 frames) due to motion blur during rapid gesture execution. To mitigate these false negative gaps, the dataset preprocessing pipeline implements a linear interpolation algorithm. When a consecutive sequence of missing landmarks falls below a defined frame threshold , e.g., a maximum gap of four frames,, the system mathematically estimates the hand's location by linearly interpolating the spatial coordinates between the last known and next known valid frames. This algorithmic imputation was hypothesized to computational bridge minor motion blur interruptions, theoretically providing the downstream temporal classifier with a continuous, noise-free kinematic sequence. The empirical efficacy of this preprocessing strategy is evaluated in Section 5.2.1.

3.4 Decoupled System Architecture

The core methodology of this research relies on the strict decoupling of spatial and temporal feature extraction within the gesture recognition pipeline. Conventional approaches often process raw video streams via monolithic 3D-CNN end-to-end architectures, which demands immense memory bandwidth and processing power [66]. This renders them unsuitable for low-power edge robotics such as the NVIDIA Jetson AGX Orin. By contrast, this work implements a cascaded information compression strategy. This process progressively increases information density while simultaneously reducing the dimensionality and memory footprint of the input data at each stage of the pipeline.

The pipeline architecture, illustrated in Figure 3.1, is composed of four distinct stages:

- **High-Redundancy Raw RGB Input:** The system initially ingests high-dimensional RGB streams. This data is characterized by significant spatial and temporal redundancy, including static background textures, fluctuating environmental lighting, and irrelevant pixel variations, all of which do not contribute to gesture classification.
- **ROI Cropping:** To mitigate background noise, the pipeline performs mathematical abstraction by isolating the hand region. Pixels external to the hand's bounding box are discarded, reducing the computational domain.
- **Spatial Landmark Extraction via Skeletal Reduction:** The ROI is processed to extract 21 anatomical 3D coordinates. This stage achieves a significant data reduction: raw pixel-based input is transformed into a compact 63-dimensional vector (21 joints $\times$ 3 coordinates). This vector retains the necessary kinematic information while discarding pixel-level visual data [66].
- **Temporal Classification:** The system aggregates extracted landmark vectors over a sliding window of $T = 40$ frames, forming a 40×63 temporal matrix. This matrix is processed by a bidirectional GRU network, which maps the movement trajectory to the defined gesture classes.

Because the temporal GRU operates exclusively on normalized landmark coordinates, the model is inherently invariant to visual context. It remains robust to background clutter, room lighting, and camera hardware discrepancies [21]. This independence allows the system to be trained on datasets such as the 20BN-Jester dataset, which features heterogeneous indoor recording environments, and subsequently deployed on robotic platforms without experiencing performance degradation due to visual domain shifts.

3.5 Spatial Feature Extraction via MediaPipe

The primary objective of the spatial feature extraction stage is to transform raw input frames into a structured, noise-resilient skeletal representation. This process is executed via the MediaPipe framework, which is widely considered state-of-the-art for real-time, on-device hand tracking [67]. This status is empirically supported by its ability to achieve sub-centimeter coordinate precision, maintaining consistent real-time inference latency below 15 ms per frame on constrained mobile and edge hardware. Clinical evaluations of its tracking accuracy report a mean root-mean-square error of 0.28 normalized pixels when compared against touchscreen ground truth data [63]. Training a custom spatial landmark detector was deemed infeasible for this architecture, primarily because the 20BN-Jester dataset provides only video-level gesture classifications, completely lacking the dense 3D coordinate ground truth required for spatial supervision. Furthermore, even if a distinct dataset with landmark annotations were utilized for training, a custom spatial extraction model trained on constrained academic datasets is highly improbable to match the robustness and generalization capabilities of MediaPipe. This production-grade

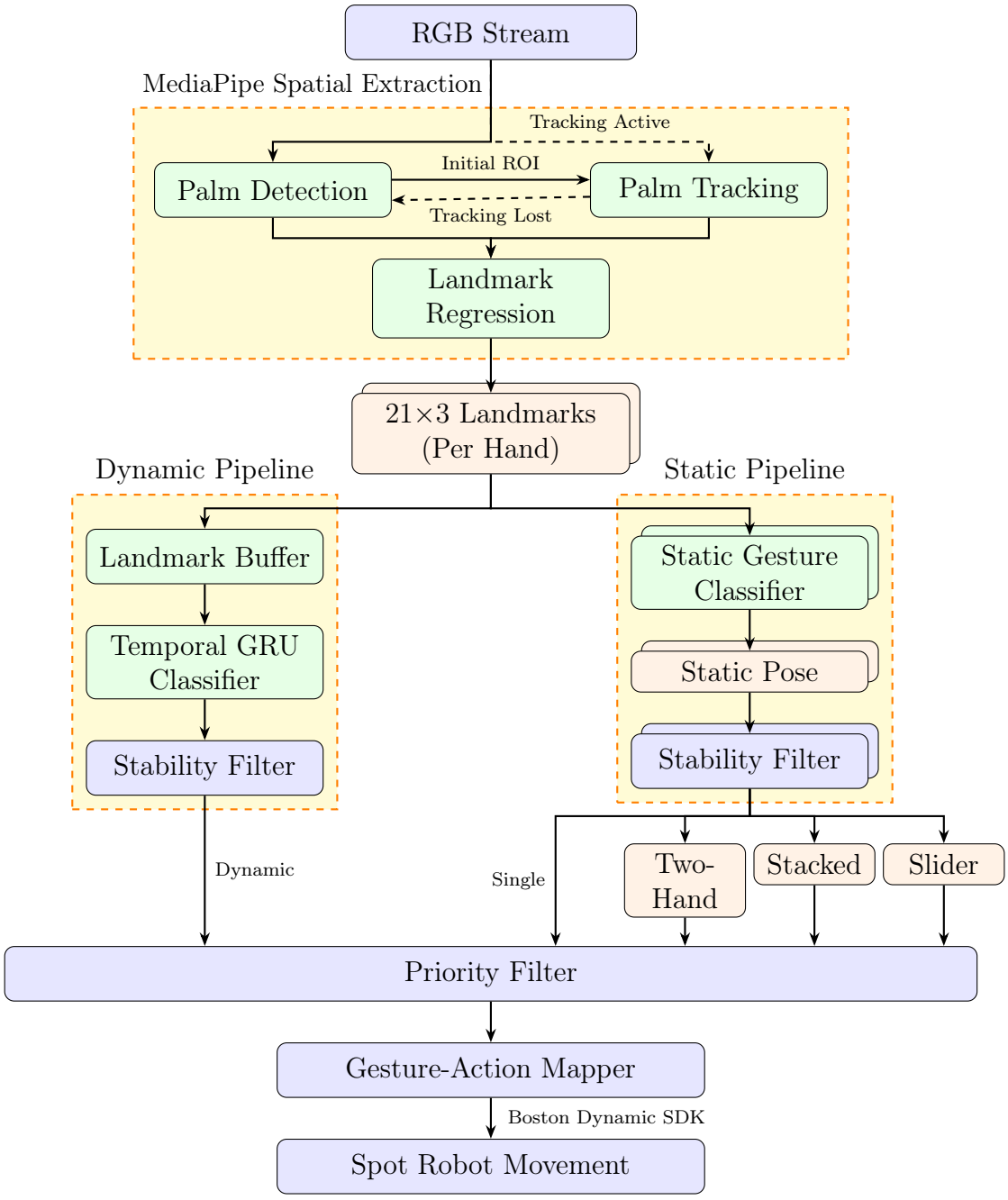


Figure 3.1: End-to-end architecture of the gesture control pipeline, detailing the continuous data flow from the raw RGB webcam stream to the final Boston Dynamics robot commands. The pipeline features parallel paths for static and dynamic classification, synchronized via left/right hand logic.

framework leverages extensive, proprietary datasets curated by Google to ensure environment-invariant tracking.
Crucially, relying on an established framework mitigates the risk of introducing algorithmic bias into the robotic control pipeline. Google's internal fairness evaluations of the MediaPipe hand tracking model across six distinct skin tones on the Fitzpatrick scale demonstrate a highly consistent Mean Normalized Absolute Error of 5.08% with a marginal standard deviation of 0.72%. Similarly, evaluations across genders yielded a stable Mean Normalized Absolute Error of 5.09% with a standard deviation of 0.29%. These metrics empirically confirm that the tracker maintains equitable performance regardless of the operator's demographic characteristics [20]. This is a critical ethical consideration when deploying autonomous robotic systems in diverse public or industrial environments.
This architecture relies on two distinct internal processes:

- **Palm Detection:** An object detection model that localizes the hand regions within the frame. It operates on a downsampled input resolution (192×192) to achieve global localization. Because this model processes the entire image, it acts as the primary computational bottleneck and the most time-consuming component of the spatial extraction load [67].
- **Temporal Tracking Algorithm:** To maintain real-time performance on edge hardware, the system bypasses the expensive Palm Detector during continuous video processing. Instead, it utilizes temporal coherence by deriving the current frame's bounding box from the hand landmarks extracted in the preceding frame. The full Palm Detector is only re-triggered if the tracking confidence falls below a critical threshold or if the hand is lost entirely [67].

- **Hand Landmark Tracking:** Operating on the bounding box provided by either the Palm Detector or the Temporal Tracking Algorithm, a lightweight regression model estimates the 21 anatomical landmarks. This stage processes a higher-resolution cropped region-of-interest (256×256) to extract precise 3D coordinates [67].

By strictly limiting the configuration to a maximum of two hands, the system prioritizes the stability of the Landmark Tracker over the computational overhead required to continuously detect new hands entering the frame. If the tracker were configured to search for an arbitrary number of hands, the pipeline would be forced to frequently re-trigger the Palm Detector to scan the entire image for newly introduced operators. This constant global searching would increase computational load. By capping the configuration at two hands, the system relies almost exclusively on the efficient Temporal Tracking Algorithm for ongoing interactions, maintaining a consistent frame rate by bypassing the expensive full-frame Palm Detector.
A specific characteristic of this detection logic is its reliance on confidence-based prioritization, which inherently favors high-probability candidates such as hands in the foreground. This behavior is intentionally exploited to ensure that the robotic platform is not falsely triggered by individuals moving in the background while the primary operator interacts in the foreground. If the currently tracked hands transition from the foreground to the background while a new pair enters the foreground, the system persists in tracking the initial pair. The tracker will only transition to the new foreground candidates once the tracking of the original pair is lost. While this behavior preserves temporal consistency for ongoing interactions, it requires clear operator positioning to avoid unintended hand switching.

3.6 Temporal Classification via GRUs

Following the spatial extraction of 3D anatomical landmarks, the skeletal time series is ingested by the temporal classification model. Recent research highlights a paradigm shift away from computationally demanding, end-to-end 3D-CNNs toward decoupled pipelines that pair skeletal tracking with recurrent layers. This approach captures both spatial and temporal sequences effectively without the substantial data-augmentation overhead required for processing raw video [6, 50].

3.6.1 Skeletal Augmentation

To ensure the model generalizes effectively across diverse robotic deployment scenarios, the raw landmark data undergoes an extensive augmentation phase during training. Relying solely on raw coordinates can cause the model to overfit to specific pixel locations rather than learning the underlying kinematic motion. As illustrated in Figure 3.2 and Figure 3.3, the spatial and scale distribution of operator hands within the Jester dataset is heavily concentrated in specific regions and distances. This bias originates from the dataset's collection methodology. Participants recorded their gestures while sitting directly in front of a laptop webcam. Consequently, the

hands are naturally positioned near the center of the frame and at a relatively close proximity to the camera. In contrast, an operator communicating with a robotic platform via hand gestures typically stands at a significantly greater physical distance and may issue commands from various angles within the field of view.

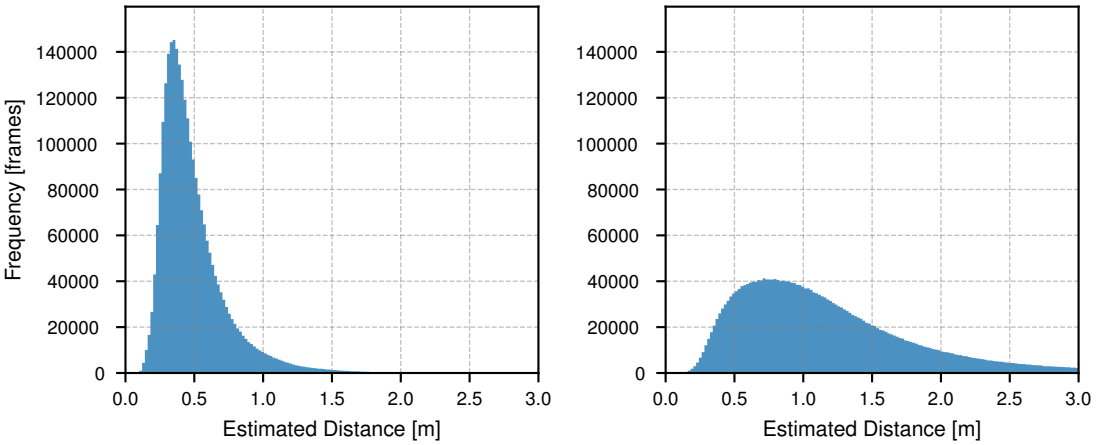


Figure 3.2: Estimated distance distribution of operator hands across the full filtered Jester dataset, containing $2,698,264$ *detected hand frames. The physical distance in meters is mathematically approximated from the normalized hand diagonal. Assuming an average human hand diagonal of* 22 *cm and a typical webcam diagonal field of view of* $75°$*, the distance is calculated by scaling the inverse of the normalized bounding box diagonal by a constant factor of* 0.2*. This conversion provides a significantly more intuitive metric for robotic teleoperation. For example, raw 2D scales, such as a hand occupying* $1/8$ *of the screen diagonal, are difficult to physically interpret and scale non-linearly compared to real-world meters. The original distribution shows a strong concentration around* $0.5m$*, reflecting operators sitting at a desk, while the simulated distribution following physics-based augmentation extends the distances significantly to simulate standing operations.*

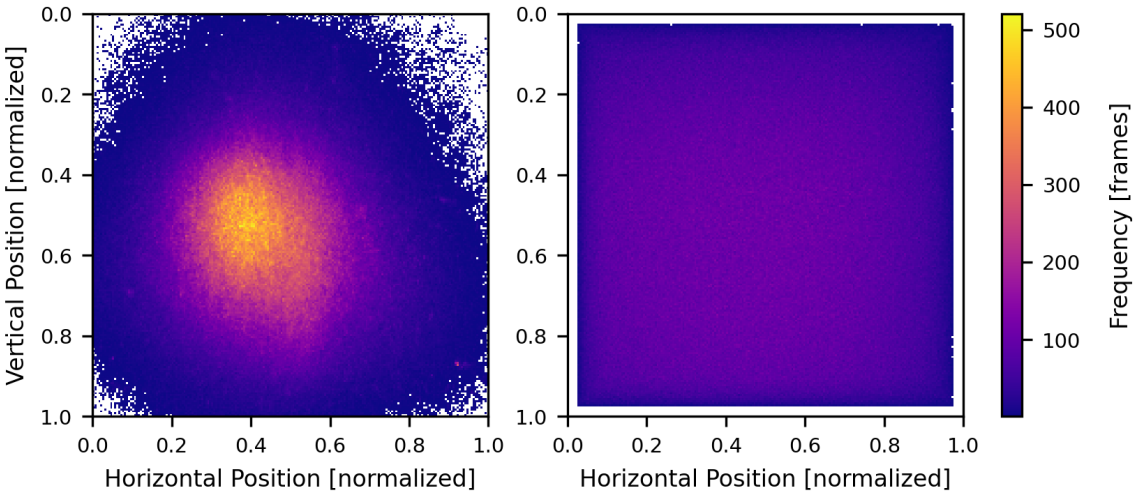


Figure 3.3: Spatial distribution of operator hands across the full filtered Jester dataset, containing $2,698,264$ *detected hand frames. The original spatial distribution shows a dense concentration in the center regions. A slight bias toward the left side of the frame is also observed, which correlates with the global demographic majority of right-handed individuals, estimated at* 89.4% *[48], as the right shoulder naturally sits on the left side of an unmirrored webcam feed. This demographic bias is explicitly confirmed by the heavily skewed right-hand classifications extracted from the dataset (Figure 3.4). Following uniform scaling and bounded shifting augmentation, the simulated spatial distribution is heavily expanded. The dynamic bounding logic prevents any landmark from being shifted off-screen, creating a visible buffer zone that keeps the centroid away from the edge by a distance equal to the physical radius of the hand.*

This demographic bias is explicitly confirmed by the handedness distribution extracted from the dataset, which reveals a heavily skewed ratio favoring right-hand executions (Figure 3.4).

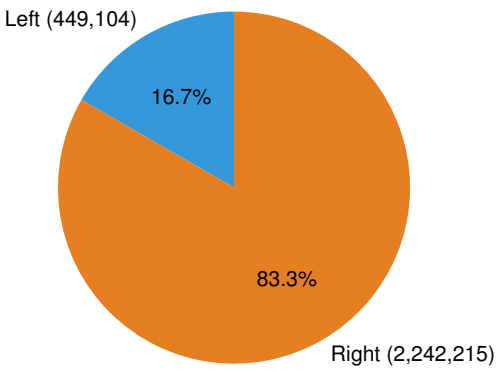


Figure 3.4: Handedness distribution across the Jester dataset. The significant majority of right-hand classifications aligns with the global demographic bias (estimated at 89.4% *[48]) and the observed spatial concentration on the left side of unmirrored webcam feeds.*

To counteract this potential spatial bias and limited distance range, the training pipeline dynamically applies two distinct augmentation strategies to the 40×63 temporal matrix:

- **Physics-Based Scaling:** Because perceived 2D scale is inversely proportional to physical distance in a pinhole camera projection, sampling a uniform 2D scale factor directly would fail to simulate a realistic and uniform shift in physical distance. To accurately simulate an operator stepping further away from the robot, a physical distance multiplier $k \sim \mathcal{U}(1.0, 4.0)$ is first sampled uniformly per video sequence. This physically grounded approach drastically increases the overall operating distance and spreads the dataset distribution over a significantly wider range, guaranteeing a substantially more diverse spatial representation for the model to train on. The corresponding 2D visual scale factor $s = 1.0/k$ is subsequently calculated and applied to all spatial coordinates relative to their geometric centroid.
- **Spatial Shifting:** A random translation offset dx and dy, drawn from a uniform distribution $\mathcal{U}(-0.5, 0.5)$, is uniformly added to the x and y coordinates of every landmark across all frames. The depth coordinate z remains unaltered. To preserve data integrity, the maximum allowable shift is dynamically bounded per sequence to guarantee that no augmented landmark exceeds the normalized camera frame limits of 0.0 and 1.0. Because this safety logic evaluates the entire bounding box rather than just the centroid, a natural buffer zone emerges at the edges of the spatial distribution. The centroid is mathematically prevented from reaching the absolute edge of the frame by a distance equal to the radius of the hand, ensuring no structural data is lost off-screen. This simulates the hand appearing at different physical locations within the field of view, ensuring the classifier remains translationally invariant without training on corrupted skeletal topologies.

A zero-padding mask is strictly maintained during these transformations to ensure that artificially padded frames in sequences shorter than 40 frames are not inadvertently shifted into active coordinate space.

3.6.2 Bidirectional GRU Architecture

The core temporal classifier is implemented as a bidirectional GRU. While LSTM networks are traditionally used for sequential data, GRUs were explicitly selected for their structural simplifications. A GRU cell relies on only two gates and merges the internal cell state with the hidden state. This architectural simplification reduces the total parameter count by approximately 25% compared to an LSTM, yielding a significantly lighter and more energy-efficient model [7, 52]. Empirical hardware benchmarks demonstrate that this reduced footprint translates directly to superior computational efficiency on edge devices, notably decreasing GPU memory allocation and access times while maintaining equivalent classification accuracy [11].

Furthermore, the theoretical advantage of LSTMs over GRUs is generally observed only on extremely long data sequences, often exceeding 1000 temporal instances. Empirical consensus shows that for short temporal windows, typically under 300 frames, GRUs match or outperform LSTMs while actively mitigating the risk of overfitting in smaller data regimes [9]. Because the robotic gesture pipeline strictly limits the temporal window to 40 frames, the GRU is the optimal choice for ensuring robust, real-time inference on the Jetson edge architecture.
To maximize efficiency on the edge hardware, the model is heavily scaled down to a hidden dimension of just 32 units across 3 stacked layers. To combat overfitting within this compact architecture, dropout layers with a probability of 0.3 are interleaved between the recurrent blocks. The bidirectional nature of the network allows it to process the 40-frame sequence both forward and backward in time, capturing contextual motion dynamics from both temporal directions. Finally, the sequential output is temporally aggregated using mean-pooling across the frame dimension, compressing the sequence into a fixed feature vector before classification via a fully connected head (Figure 3.5).

3.7 Hybrid Architecture: Static Gestures and Complex Interactions

While the bidirectional GRU effectively captures the complex temporal dynamics required for discrete gesture commands, such as swiping or waving, a purely dynamic classification pipeline inherently introduces latency. Specifically, the necessity of buffering a 40-frame temporal window prior to inference imposes an absolute minimum latency barrier. For safety-critical robotic teleoperation, this delay is infeasible for emergency interrupt signals. Furthermore, a recurrent model classifying a discrete motion sequence cannot natively facilitate continuous, high-resolution parameter adjustments, which require real-time spatial tracking rather than post-hoc temporal classification.
To resolve these architectural limitations, a parallel static gesture classification system was integrated alongside the primary dynamic GRU model, as depicted in the static pipeline branch of Figure 3.1. This hybrid approach leverages the out-of-the-box static pose recognition capabilities of the MediaPipe framework to evaluate single-frame spatial coordinates instantaneously. By routing the spatial data stream to both classifiers simultaneously, the system achieves a multi-modal control interface capable of zero-latency safety overrides and complex, continuous teleoperation.

3.7.1 Static Gesture Classifier

The static gesture classifier operates exclusively on the spatial coordinates of the current frame, bypassing the temporal buffer entirely. By analyzing the relative distances, joint angles, and skeletal topology of the 21 3D landmarks, the system deterministically identifies fixed hand poses. Specifically, the out-of-the-box MediaPipe Gesture Recognizer [39, 18] was integrated, which classifies seven pre-trained

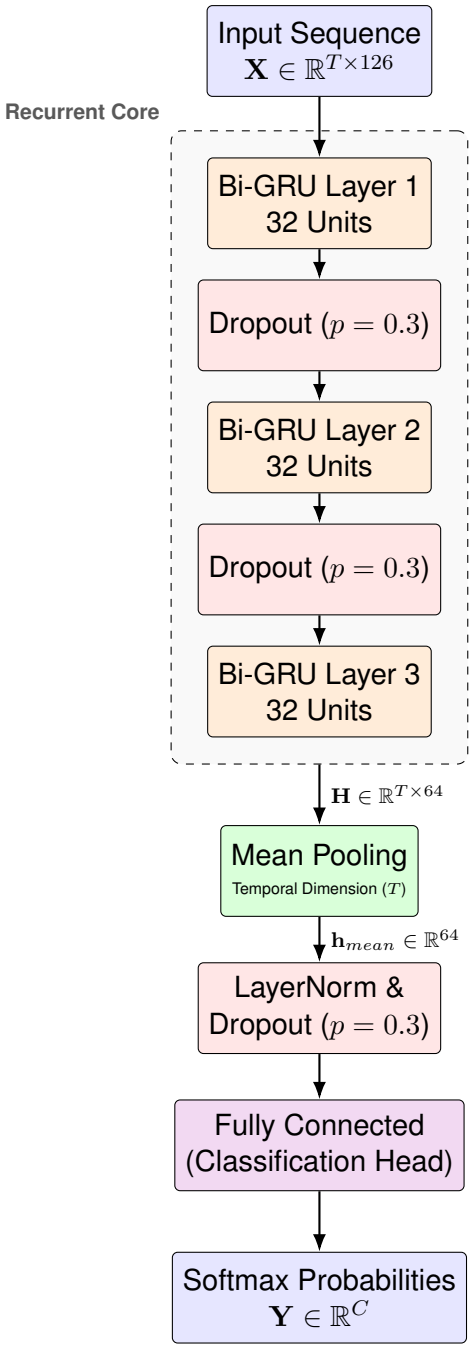


Figure 3.5: Network architecture diagram of the custom temporal classifier. The pipeline features a 3-layer Bidirectional GRU with 32 hidden units per direction, interleaved dropout regularization ($p = 0.3$), followed by temporal mean-pooling, LayerNorm, a final dropout layer, and a fully connected classification head mapping to the gesture probabilities.

foundational static gestures: *Closed Fist*, *Open Palm*, *Pointing Up*, *Thumb Down*, *Thumb Up*, *Victory*, and *Metal*.
Because this classification requires no long-term temporal context, it provides a rapid, frame-by-frame state evaluation. While the raw spatial extraction incurs a marginal hardware inference latency, reported in literature to be < 20 ms on edge central processing units (CPUs) [18], the required temporal stability smoothing introduces a small algorithmic delay. However, this delay is effectively imperceptible and remains orders of magnitude faster than the large 40-frame buffer required for the dynamic GRU. This low-latency capability is explicitly exploited to trigger safety mechanisms. For example, recognizing an open palm near-instantaneously overrides all pending dynamic commands and executes an Emergency Stop (E-Stop) on the robotic platform, ensuring rapid compliance in hazardous situations.

3.7.2 Two-Hand Gestures

The integration of the static classifier naturally enables the simultaneous tracking and classification of two distinct hands. Rather than forcing the dynamic model to learn complex, bimanual temporal sequences, which would drastically increase the dataset size and training complexity, the system combines the static states of both hands to form composite commands. This bimanual logic expands the command matrix, allowing operators to execute a broader vocabulary of instructions simply by combining recognized static poses across two hands. Furthermore, requiring the simultaneous coordination of both hands intrinsically acts as a physical safeguard; bimanual gestures are significantly less prone to being triggered unintentionally during natural, everyday human motion compared to single-hand gestures.

3.7.3 Continuous Parameter Control

Dynamic gestures are inherently discrete, returning a single classification upon the completion of a motion. To facilitate continuous parameter control, such as dynamically adjusting the robot's walking speed or chassis height, the system utilizes a distance-based bimanual slider mechanism. This modality is engaged when both hands simultaneously hold a specific static gesture, acting as a modality lock.
Upon recognition of this dual-hand pose, the system transitions into a continuous tracking state. The initial Euclidean distance measured between the two hands at the exact moment of engagement defines the baseline, representing the current floating-point value of the controlled parameter. Subsequent spatial tracking continuously measures the relative change in distance between the operator's hands. For instance, decreasing the physical distance by 50% dynamically reduces the corresponding parameter value by 50%. Because this slider mechanism operates strictly on the frame-by-frame output of the spatial classifier, it facilitates real-time parameter manipulation. Operators can visually perceive the robotic platform modifying continuous states, such as adjusting its chassis height, near-instantaneously in response to physical hand displacement.

Because the manipulation is relative to the starting distance, operators must physically plan their initial hand placement based on the desired adjustment. Executing a large scalar increase, such as multiplying the value by a factor of three, requires engaging the gesture with the hands initially positioned close together to ensure sufficient physical space for expansion within the camera frame. Furthermore, this mechanism supports a clutching behavior. The operator can release the static gesture to temporarily disengage the slider, reposition their hands, and re-engage the pose to redefine the baseline distance without altering the underlying parameter value.

3.7.4 Stacked Gestures

To further increase the dimensionality of the teleoperation interface without requiring a larger neural network vocabulary, the architecture supports stacked gestures. In this modality, an operator performs a predefined temporal sequence of distinct gestures using the same hand one after another.
Furthermore, requiring a precise temporal chain of specific classifications intrinsically acts as a physical safeguard. Stacked gestures are significantly less prone to being triggered unintentionally during natural, everyday human motion compared to isolated single-action commands. However, this robustness introduces a strict human-computer interaction trade-off. Sequential gestures physically require more time to execute than single actions, and they significantly increase the cognitive load placed on the operator, demanding higher memorization complexity to utilize effectively.

3.8 Pipeline Filters and Conflict Resolution

While the dynamic GRU inherently aggregates sequential data, it can still produce momentary misclassifications at the boundaries of gestures. Furthermore, the frame-by-frame nature of the static classifier makes it highly susceptible to flickering caused by sensor noise, motion blur, or overlapping gesture boundaries. To ensure deterministic and safe robotic actuation, two distinct filtering layers were implemented to resolve the raw outputs of the entire pipeline.

3.8.1 Temporal Stability Filter

To prevent the robotic platform from executing rapid, oscillating commands when classifications fluctuate, the architecture shown in Figure 3.1 applies a Temporal Stability Filter [55] to both the spatial and temporal output streams. This filter employs a sliding temporal buffer of size N to track the recent history of inferred gestures.
A gesture is only accepted if its frequency within the buffer exceeds a predefined ratio threshold. Crucially, empty frames or failed detections are included in this buffer to actively dilute the stability score, ensuring that sparse or uncertain detections cannot

trigger an action. Furthermore, a hysteresis mechanism is enforced during state transitions: a newly recognized gesture must dominate the buffer for a fixed number of consecutive frames before the system officially accepts the state change. This effectively suppresses frame flickering and ensures that only deliberate, sustained gestures are passed to the controller.

3.8.2 Priority Filter

Because the hybrid architecture tracks diverse gesture modalities simultaneously, conflicting command states frequently arise. Inherently, complex multimodal gestures contain simpler gestures within their physical execution. For example, while an operator performs a bimanual composite gesture, the system theoretically detects two valid, isolated single-hand static poses simultaneously. Without arbitration, the system would erroneously execute both constituent modular sub-commands concurrently rather than the intended unified composite action.
To resolve this, the architecture illustrated in Figure 3.1 implements a strict hierarchical Priority Filter based on gesture complexity. The fundamental rationale is that while an operator might accidentally trigger a basic one-hand pose while setting up a complex interaction, it is physically improbable to accidentally trigger a highly coordinated bimanual or dynamic sequence while intending to perform a simple static pose. Therefore, the filter strictly prioritizes more complex modalities over simpler ones. The defined hierarchy is executed in the following descending order: Dynamic Gestures, Stacked Gestures, Continuous Sliders, Two-Hand Static Combinations, and finally, isolated Single-Hand Static poses. By enforcing this hierarchy, the system ensures that complex intents reliably intercept and override their constituent modular poses.

3.9 Gesture-to-Action Mapping

Once a gesture is stabilized and processed by the filters, it is mapped to a concrete robotic instruction via the Boston Dynamics software development kit (SDK). To prevent the blocking of the real-time inference loop during hardware communication, each actuation command is dispatched asynchronously to a daemonized worker thread.

3.9.1 Pre-Filtering

To minimize computational overhead and prevent unnecessary conflict resolution, the system employs an early-discard mapping strategy. Any recognized gesture that does not have a corresponding action explicitly defined in the mapping dictionary is immediately discarded before it reaches the Priority Filter.
This pre-filtering mechanism introduces a critical design constraint for teleoperation safety: operators should avoid defining single-hand static actions using the same gesture classes that constitute their complex bimanual or stacked commands. For

example, if a bimanual composite gesture requires an operator to hold a peace sign with their left hand, the isolated left-hand peace sign should ideally remain unmapped. By leaving the constituent pose unmapped, the system automatically discards it during the setup phase of the bimanual action, entirely avoiding the need for the Priority Filter to resolve a potential conflict. This drastically reduces the probability of false-positive triggers and streamlines the execution pipeline.

3.9.2 Safety Architecture

A critical requirement of teleoperation is the ability to rapidly halt the system. Within the mapping dictionary, specific static gestures are hardcoded to high-priority safety interrupts. As established in Section 3.7, the Open Palm gesture is globally mapped to the E-Stop command. Because it is evaluated by the low-latency static classifier, the E-Stop strictly bypasses the algorithmic delay of the dynamic GRU and the timeout window of the Priority Filter. While it remains subject to the minimal sliding window of the Stability Filter to prevent false-positive triggers, the execution pipeline implements three unique architectural mechanics specifically for the stop action to guarantee a near-immediate hardware interrupt.
First, the stop action inherently bypasses the active execution lock. While standard incoming commands are explicitly blocked if the system is currently executing a prior action, the stop command overrides this logic to guarantee interruption. Second, rather than dispatching the command asynchronously to a daemonized worker thread such as standard actions, the stop command is executed synchronously within the main inference thread to minimize latency. Finally, the stop action executes a forced state clearing routine. It proactively purges the active action state memory, effectively killing the execution of the previously running command before halting the robotic platform's motor controllers.

3.10 Baseline Architectures and Benchmarking

To systematically evaluate the efficacy and computational efficiency of the proposed champion architecture, a comprehensive benchmarking study was formulated. Relying solely on absolute performance metrics provides insufficient scientific context. A model must be evaluated against established alternative paradigms to quantify its specific architectural trade-offs.
The evaluation framework isolates the spatial extraction phase and the temporal classification phase, benchmarking the champion pipeline against state-of-the-art baselines spanning convolutional, recurrent, graph-based, and attention-based paradigms. Finally, the decoupled skeletal approach is benchmarked against a monolithic, end-to-end video processing baseline.

3.10.1 Spatial Extraction Baseline

The champion architecture utilizes MediaPipe Hands [39] for spatial extraction, relying on a highly optimized, two-stage pipeline consisting of palm detection and

localized landmark regression. To evaluate whether a denser, single-stage convolutional architecture offers superior spatial robustness, particularly against motion blur, You Only Look Once (YOLO) pose estimation was selected as the spatial baseline. Because YOLO processes the entire image simultaneously without relying on an intermediate ROI crop, the hypothesis is that its global contextual awareness yields greater robustness to rapid hand movements.
However, hand landmark detection remains a highly niche domain. Even the prominent Ultralytics framework does not provide an official pretrained hand pose model, only offering full-body and facial keypoints. Consequently, a community-contributed YOLO11 hand pose model fine-tuned by the developer *xutom* [65] was utilized. Although the community repository retains the legacy nomenclature *YOLOv8-Hand-Pose*, the underlying weights used in this work leverage the more recent YOLO11 architecture. A major limitation of this approach is dataset availability. Because Google's MediaPipe training data is proprietary, community models such as this YOLO variant are trained on data pseudo-labeled by the MediaPipe model itself. This establishes a knowledge distillation paradigm [24] where the YOLO student model is theoretically bounded by the capabilities of its MediaPipe teacher. Nevertheless, the profound structural differences between the architectures may foster emergent generalization, allowing the dense single-stage network to outperform its teacher in specific edge cases such as motion blur. The primary drawback of this baseline is computational efficiency. YOLO requires significantly more parameters and FLOPs to process the full uncropped frame compared to MediaPipe's lightweight ROI-bound regression.

3.10.2 Temporal Skeletal Baselines

To evaluate the selected Bidirectional GRU, four alternative sequence modeling architectures were implemented to process the extracted 3D landmark arrays.
1D Convolutional Neural Network: Treating the temporal sequence of spatial coordinates as a multi-channel 1D signal, a 1D convolutional neural network (1D-CNN) [27] applies temporal convolutions to extract local motion patterns. The primary advantage of 1D-CNNs is their highly parallelizable architecture, enabling extremely rapid inference without recurrent bottlenecks. This serves as a non-recurrent baseline to evaluate whether explicit temporal memory is strictly required, or if basic hierarchical pooling over the temporal dimension is sufficient for dynamic gesture classification.
LSTM Network: The LSTM [25] represents the classic recurrent baseline. Designed specifically to mitigate the vanishing gradient problem, the LSTM employs a complex gating mechanism utilizing a separate cell state and a dedicated forget gate to retain long-term dependencies. Benchmarking against the LSTM empirically determines whether this added parametric complexity yields superior temporal understanding or simply introduces unnecessary computational overhead for short, 40-frame gesture sequences compared to the lightweight GRU.
ST-GCN: The ST-GCN [66] represents the modern academic standard for skeleton-based action recognition. Rather than processing the hand coordinates as a flattened

feature vector, The ST-GCN explicitly models the biological topology of the hand as a graph, where joints act as nodes and physical bones act as edges. Its core advantage is the ability to learn highly structured, biomechanically aware spatiotemporal representations. This baseline evaluates whether explicitly encoding human anatomy natively outperforms the unstructured feature processing of the champion GRU.
Transformer: To evaluate the efficacy of attention mechanisms, a standard Transformer [62] was selected as a baseline. The profound advantage of this architecture is its global receptive field, allowing it to correlate any two frames regardless of their temporal distance. While representing the modern standard for sequence modeling, the Transformer possesses significantly more parameters and scales quadratically with sequence length, making it less standard for resource-constrained edge computing. This benchmarks whether global temporal attention provides higher classification accuracy than the sequential, step-by-step memory of lightweight recurrent networks despite the increased computational overhead.

3.10.3 End-to-End Processing Baseline

The fundamental premise of the champion architecture is decoupling the pipeline into spatial extraction followed by temporal classification. To challenge this core assumption, the system is benchmarked against a monolithic, end-to-end baseline: a 3D-CNN. Specifically, a Deformable ResNeXt3D architecture [22] was selected.
Rather than relying on intermediate skeletal representations, the 3D-CNN directly processes the raw RGB video volume, convolving simultaneously across spatial width, height, and temporal depth. The primary advantage of this paradigm is that it learns optimal spatiotemporal features directly from raw pixels, eliminating any critical information loss caused by the skeletal extraction bottleneck. While mathematically capable of modeling highly complex environmental interactions without relying on fallible external landmark detectors, this approach is notoriously computationally expensive. This baseline serves as the ultimate hardware constraint evaluation, quantifying the exact sacrifices in edge-device inference latency and memory footprint required to achieve monolithic end-to-end classification.

3.11 Summary

In summary, this chapter established the core theoretical and architectural foundations for the proposed gesture recognition system. By strictly decoupling the pipeline into spatial landmark extraction via MediaPipe and temporal classification via lightweight GRUs, the architecture mathematically mitigates the computational bottlenecks associated with traditional monolithic 3D-CNNs. The methodology detailed the exact preprocessing steps, the hybrid classification mechanics for resolving static versus dynamic inputs, and the specific mapping strategies required to translate human motion into actionable robotic commands.

4
Implementation & Experiments

This chapter details the technical implementation of the gesture control pipeline, bridging the theoretical foundations established in Chapter 3 with the practical software engineering required for edge deployment. The complete source code for the proposed architecture, including the robot control application programming interface (API) and evaluation scripts, is publicly available at `https://github.com/Bennit99/robot-gesture-control`.

4.1 Hardware Setup

The system is deployed on a Boston Dynamics Spot robot. Spot is an agile, quadrupedal platform designed for autonomous and teleoperated navigation in unstructured environments. Unlike traditional wheeled robots, Spot's quadrupedal locomotion enables it to reliably traverse complex terrains, including stairs, dense forests, gravel, and steep mountain slopes [4]. The platform is leveraged for two primary domains of application. First, in industrial use cases, it performs routine maintenance, automated site inspections, and monitoring of critical infrastructure, such as the autonomous inspection of the Joint European Torus (JET) fusion reactor [4, 57]. Second, the platform is highly prominent in contemporary scientific research, serving as a versatile mobile testbed for developing advanced autonomous behaviors. Recent studies leverage Spot to explore hazardous environments by outfitting the chassis with specialized thermal imaging and depth cameras. This hardware integration directly enables fire-aware navigation [64]. Despite its versatility, direct HRI in the field remains challenging due to the reliance on traditional tablet controllers.

To facilitate edge-based gesture recognition, the compute architecture is powered by an NVIDIA Jetson AGX Orin Developer Kit (64GB), which is physically mounted on the back of the Spot robot. This dedicated edge device is engineered specifically for advanced autonomous robotics, providing up to 275 tera operations per second of AI performance within an efficient 15W to 60W power envelope. The NVIDIA Jetson AGX Orin operates in configurable power modes. To establish theoretical baseline capabilities during inference benchmarking, the system is configured to its maximum performance profile, known as MAXN mode. In this mode, CPU and GPU clock frequencies are explicitly pinned to their maximum values to eliminate runtime variance. The system-on-module utilizes a powerful Advanced RISC

Machines (ARM) architecture (12-core Cortex-A78AE). This design is based on a reduced instruction set computer (RISC) paradigm, achieving significantly higher energy efficiency than traditional x86 CPUs through its streamlined instruction set and responsive dynamic clock scaling. For visual processing, it features a 2048-core NVIDIA Ampere GPU equipped with 64 Tensor cores and 64GB of 256-bit LPDDR5 video random access memory (delivering 204.8 gigabyte (GB)/s memory bandwidth). This substantial compute density provides abundant hardware acceleration, supplying a substantial overhead sufficient to execute both the spatial feature extraction and temporal classification models in real-time without severely draining the robot's primary locomotion battery.
While Spot's built-in stereo cameras provide a 360-degree field of view, they are mechanically angled downwards to support the robot's internal collision avoidance and mapping algorithms. Consequently, they only capture the legs and lower torsos of nearby humans, rendering hand gesture recognition physically impossible in a default stance. Although the robot can be commanded to pitch its chassis upwards to force the internal cameras to view a human's upper body, this workaround is highly impractical. Requiring an operator to manually tilt the robot via tablet, or forcing the system to preemptively anticipate a gesture before observing the user, completely defeats the purpose of a spontaneous, hands-free interface. To overcome this fundamental hardware limitation, the system relies on a dedicated external RGB webcam (Figure 4.1). Mounted on the back of the robot at an optimal upward angle, this external camera provides a continuous stream of the operator's upper body, ensuring the robot is always ready to receive spontaneous commands. However, the software architecture is inherently camera-agnostic. Through modular configuration, the pipeline can seamlessly ingest video streams from internal cameras if desired. This flexibility ensures the framework is highly transferable to humanoid platforms, where integrated head-mounted cameras naturally reside at the correct ergonomic height for gesture recognition.

4.2 Training Infrastructure & Dataset Implementation

To train the temporal classification models, the 20BN-Jester dataset [41] was utilized. The raw video files were preprocessed into a standardized spatial format using the MediaPipe framework. These extracted 3D spatial features were systematically cached to disk as Parquet files to eliminate video decoding bottlenecks during the training loop.
Based on the officially provided dataset splits, the `JesterLandmarkDataModule` ingested exactly 118,562 videos for the training set and 14,787 videos for the validation set, batching them into tensors for rapid GPU transfer.



Figure 4.1: Hardware configuration of the Boston Dynamics Spot robot. The left highlight denotes the external RGB webcam mounted on the payload rails, and the right highlight indicates the NVIDIA Jetson AGX Orin compute module utilized for local gesture processing.

4.3 Spatial Feature Extraction

The spatial extraction pipeline heavily dictates the theoretical bounds of the entire gesture recognition system. The champion architecture leverages the highly optimized MediaPipe framework to extract 3D skeletal landmarks.
To validate MediaPipe's efficacy against single-stage dense detectors, a community-trained YOLO11n-pose hand model [65] from the NeuralVulture repository was evaluated. The "nano" variant was specifically selected as it is the smallest and most computationally efficient model within the YOLO11 family, providing the most competitive baseline for edge deployment. Both architectures were systematically evaluated across the nearly 5.3 million frames of the Jester dataset to benchmark detection rates and computational efficiency.

4.4 Temporal Classification Models

With the spatial landmarks cached, the temporal classification models were trained using the PyTorch Lightning framework. Weights & Biases (WandB) was integrated into the training loop for comprehensive hyperparameter tracking and gradient logging.

To ensure a fair and consistent comparison across all evaluated architectures, a strictly controlled benchmark environment was established. All models were subjected to identical optimization configurations and data constraints. Training was configured for a maximum of 100 epochs with an early stopping patience of ten epochs to prevent overfitting once validation metrics plateaued. A standard batch size of 64 was utilized. The adaptive moment estimation optimizer with decoupled weight decay (AdamW) [38] was selected to provide robust regularization. Architectural regularization was further enforced via dropout [56]. Rather than enforcing a uniform dropout rate across all models, this parameter was empirically tuned to suit the inductive biases of each specific topology. Recurrent and convolutional architectures naturally hold strong spatial and temporal inductive biases, making them highly prone to memorizing the training data sequence. To counteract this lazy learning, they require a high dropout rate of 0.3 to force generalized feature extraction. Conversely, attention-based transformers lack these inherent biases and rely heavily on distributed, mathematically delicate attention weights to learn relationships from scratch. Consequently, applying excessive dropout artificially destroys the attention mechanism. Thus, a light dropout of 0.1 was strictly maintained for the transformer, aligning with established architectural standards [62]. The initial learning rate was set to 1×10^{-3} paired with a weight decay of 1×10^{-4}. A cosine annealing schedule [36] decayed the learning rate smoothly down to a minimum of 1×10^{-6} over the course of training. The optimization objective utilized cross-entropy loss configured with a label smoothing factor of 0.1 [58]. This regularization technique prevents the network from becoming overly confident in its predictions and penalizes excessively strict decision boundaries. A global gradient clipping threshold of 1.0 was applied to prevent exploding gradients. This stabilization step is especially critical for the training dynamics of recurrent neural networks.

To maintain consistent tensor dimensions, the sequence length was fixed to exactly 40 frames for every clip via standardized temporal padding and truncation. Furthermore, 16-bit mixed precision [42] was employed to accelerate training and halve the memory footprint by leveraging hardware tensor cores. This approach selectively maintains 32-bit floating-point stability where numerically necessary.

A critical engineering challenge was handling the multi-dimensional tensor layouts required by different network topologies. The base dataset module outputs a flat feature vector per frame of shape (B, T, D), where D represents the flattened Cartesian coordinates of all 21 joints. Sequence-to-sequence architectures, such as the Bidirectional GRU, LSTM, and Transformer, ingest this (B, T, D) tensor natively. However, spatial-temporal convolution and graph networks demand different dimensional orderings. For instance, the 1D-CNN dynamically transposes the input to (B, D, T) to align the temporal sequence along the convolution's spatial axis. Furthermore, the ST-GCN requires explicit joint topology. Consequently, its input is reshaped at runtime into a complex (B, C, T, V) layout, where C is the coordinate dimension and V represents the individual vertices of the hand graph.

The evaluation benchmarks the champion Bidirectional GRU against four primary alternative architectures:

- **1D-CNN:** A purely spatial-temporal convolution approach.

- **LSTM Network:** A heavier recurrent baseline.
- **ST-GCN:** A topology-aware graph model natively mapping the 21 joints of the human hand.
- **Transformer:** A standard multi-head self-attention encoder providing global receptive fields.

To ensure a fair and scientifically sound comparison across these diverse architectural paradigms, the models were evaluated along a computational Pareto frontier rather than as single, isolated instances. Following the established methodology of mapping accuracy against computational cost [59], each architecture was iteratively scaled across seven different configurations. Specifically, the total computational cost, measured in FLOPs, was approximately doubled or halved at each successive step. This computational scaling was achieved by systematically adjusting the width, or hidden layer dimensions, of the networks while keeping the network depth, or number of layers, strictly constant. This approach isolates the pure architectural efficiency, allowing for a direct comparison of which model topology yields the highest classification accuracy for a given computational constraint. Rather than relying on arbitrary bounds, the parameter sweep was conducted dynamically for each architecture. The search algorithm was initiated at a baseline computational footprint of approximately 10 mega floating-point operations (MFLOPs) and proceeded to alternatingly scale the capacity upward and downward. The downward scaling was halted when a severe drop in validation accuracy was observed. Conversely, the upward scaling was terminated when the validation accuracy reached a plateau or began to degrade due to capacity-induced overfitting. This bidirectional search concluded upon successfully yielding seven distinct network configurations. This dynamic approach organically bounds the effective operational spectrum of each evaluated model.

4.5 Pipeline Filters & Conflict Resolution

To translate raw neural network classifications into robust, real-world robotic commands, the system employs a series of specialized filters. These components are designed to suppress mechanical jitter, resolve conflicts between competing gestures, and enforce safety boundaries before transmitting instructions to the Spot robot.

4.5.1 Stability Filter

The Temporal Stability Filter, depicted as a core pipeline component in Figure 3.1, is implemented within the `GestureFilter` class, providing real-time evaluation of the incoming inference stream. The filter utilizes a highly efficient, double-ended queue acting as a sliding temporal buffer of a configured size. Rather than filtering raw skeletal coordinates or probabilistic logits, the filter directly ingests the finalized discrete gesture labels emitted by the static and dynamic classification models. As these new discrete string labels, alongside `None` values in the event of a failed hand

detection, are yielded at each frame. These labels are then appended to the queue, discarding the oldest entries.
To compute the stability score, the system isolates the most frequent gesture label currently residing within the buffer and calculates its occurrence ratio relative to the absolute maximum window size. By dividing the specific gesture count by the total buffer capacity rather than the subset of valid predictions, the algorithm inherently penalizes missing detections. Empty frames actively dilute the ratio, preventing intermittent noise from achieving the required activation threshold.
If a gesture successfully breaches the defined ratio threshold, the transition is subjected to a hysteresis check. The filter maintains a dedicated state variable tracking the previously accepted stable gesture alongside a consecutive frame counter. When the newly dominant gesture differs from the established state, the counter increments but the state transition is temporarily withheld, forcing the system to continue returning the previous gesture. Only when the new gesture sustains its dominance for a prescribed number of consecutive frames is the state transition officially authorized. This logic operates entirely in memory and adds negligible latency to the processing pipeline while effectively suppressing mechanical flickering.

4.5.2 Priority Filter

As outlined in Figure 3.1, the Priority Filter governs conflict resolution by enforcing a strict hierarchy within the `GestureActionMapper` architecture. To ensure that complex intents intercept constituent modular poses, the system evaluates incoming stable gestures in a descending order of complexity.
Dynamic macro commands are evaluated first. If a dynamic sequence is recognized by the temporal classifier, the corresponding action is dispatched immediately, preempting all static evaluations. If no dynamic gesture is present, the system evaluates static poses using a dual-state buffer mechanism.
To solve the inherent race condition where one hand is detected fractionally before the other during a bimanual gesture, the mapper implements a purposeful execution delay utilizing asynchronous `threading.Timer` instances. When an isolated single-hand gesture is detected, its execution is suspended for a configurable window of 300 milliseconds. If the complementary hand presents a valid gesture within this precise window, the timer is actively canceled, the two states are fused into a composite key, and the mapped two-hand action is executed. If the timer expires without interference, the original single-hand action is permitted to run.

4.6 Real-Time Inference Architecture & Optimizations

Deploying the theoretical models to the physical Jetson AGX Orin required strict architectural optimizations to maintain real-time throughput. The inference pipeline, depicted in Figure 4.2, bridges asynchronous background callbacks with synchronous Python execution via a thread-safe `LandmarkBuffer` and dedicated threading events.

During execution, the main loop retrieves frames from the camera and dispatches them to the asynchronous spatial extraction engine. To ensure temporal consistency, the main thread halts via a `recognizer_ready.wait()` event, synchronizing the Python environment with the background worker. Once the callback populates the `LandmarkBuffer`, the main thread resumes and executes the PyTorch temporal classification.

Crucially, to prevent hardware communication latency from bottle-necking the computer vision pipeline, the system utilizes asynchronous daemon threads for robot control. When the `GestureActionMapper` identifies a valid command, it offloads the mechanical execution to an independent background thread. This guarantees that network API calls to the physical robot never block the main thread, allowing the classifiers to continuously evaluate incoming frames without artificial latency.

To maximize throughput on the edge computing platform, the baseline PyTorch models were fundamentally restructured and compiled into highly optimized TensorRT engine binaries. Standard PyTorch graphs execute sequentially and rely on checkpoints containing raw weight matrices. Reconstructing the model architecture at runtime from a checkpoint requires the original Python source code, which introduces unacceptable execution overhead for edge deployments.

To decouple the inference engine from the PyTorch framework, the models were first exported to the ONNX format. This intermediate format creates a self-contained, hardware-agnostic computational graph that explicitly defines both the weights and the mathematical operations. The standardized graph was subsequently fed into the NVIDIA compiler to generate a highly specific Compute Unified Device Architecture (CUDA) binary, exclusively tailored to the Ampere architecture of the Jetson AGX Orin.

The TensorRT compilation process solves the latency bottlenecks through two primary optimizations. First, it performs post-training quantization to reduce the neural weights from 32-bit to 16-bit floating-point precision. This halves the memory footprint and accelerates memory access times without inducing a perceptible loss in classification accuracy. Second, the compiler executes extensive layer fusion. In standard frameworks, sequential operations such as matrix multiplications and activation functions require separate GPU kernel launches, causing expensive read and write cycles to the video memory. TensorRT fuses these consecutive nodes into a single mathematical kernel. The GPU calculates the combined output entirely within its high-speed local registers before committing the final result to memory. Furthermore, the compiler utilizes empirical kernel auto-tuning by generating hundreds of micro-kernel variations and testing them directly on the silicon. The fastest empirical variant is then permanently burned into the engine binary.

Empirical research demonstrates that this specific combination of quantization and architectural layer fusion yields inference speedups ranging from 1.8-fold to over 6.0-fold compared to native PyTorch executions, depending on the precise network topology [45].

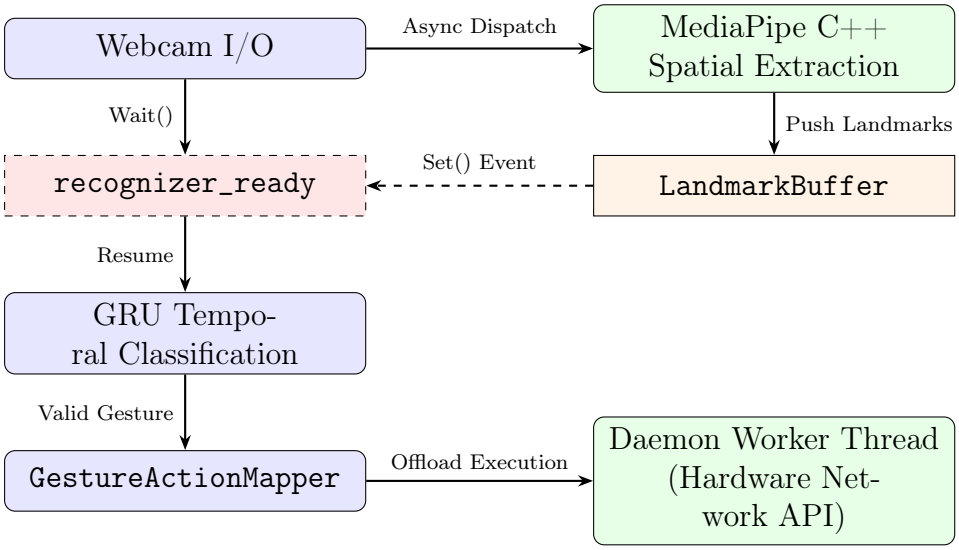


Figure 4.2: Multi-threading architecture illustrating the synchronization between the main Python thread and background execution workers to prevent artificial input latency.

4.7 Robot Control API Integration

Commanding the physical hardware requires integrating the Python-based gesture system with the proprietary Boston Dynamics SDK `bosdyn.client`.
Because Boston Dynamics enforces strict lease management to prevent conflicting commands from multiple controllers, the gesture system instantiates a `LeaseClient` configured to forcibly take control of the robot chassis. This allows the gesture interface to seamlessly override other active controllers when activated. However, an external operator (such as the standard tablet controller) retains the ultimate hardware authority to retake the lease at any time, acting as a manual failsafe. To handle this preemption safely, the system utilizes a `LeaseKeepAlive` thread equipped with an asynchronous error callback. If another client hijacks the lease, the callback immediately flags the loss of control, allowing the central streaming backend to gracefully halt operations and return the system to a safe standby state.
Spatial gestures are translated into physical motion through the `RobotCommandBuilder`. Dynamic walking gestures construct a `synchro_velocity_command`, mapping normalized scalar values to Cartesian velocity vectors v_x, v_y, and angular velocity v_{rot}. Conversely, static body-manipulation gestures invoke the `EulerZXY` orientation builder, explicitly commanding the chassis pitch, yaw, and roll while the robot remains stationary.

4.8 User Interface

While the gesture recognition architecture operates entirely headlessly, a dedicated web dashboard (Figure 4.3) was developed to bridge the gap between the autonomous system and human oversight. A video demonstration of the Web UI

dashboard and real-time skeletal tracking is available in the public project repository at `https://github.com/Bennit99/robot-gesture-control`. The backend logic is driven by a lightweight `Flask` streaming server. The frontend is built using standard `HyperText Markup Language (HTML)` and modular `JavaScript` views, styled with `Tailwind CSS` and a suite of SVG icons. In accordance with Boston Dynamics' security guidelines [3], the robotic deployment network is intentionally air-gapped from the public internet to prevent unauthorized telemetry access and ensure command integrity. Consequently, all CSS libraries, JavaScript dependencies, and icon assets are packaged and served locally to guarantee full offline functionality.
Network connectivity is established either by connecting directly to Spot's self-hosted Wi-Fi Access Point, or through a dedicated, isolated "SpotNet" router that bridges the robot and the operator interfaces. Once connected to this local subnet, operators can access the dashboard from any device with a web browser. The interface serves as a critical tool for development, debugging, and operator training by exposing the following core features:

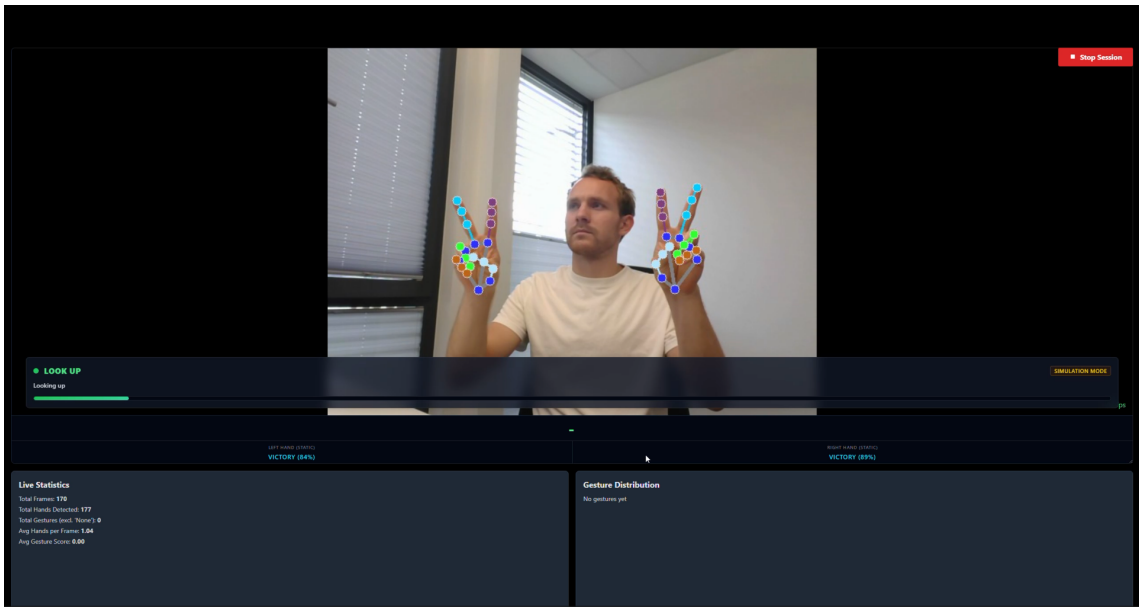
- **Visual Feedback & Confidence Scoring:** The dashboard broadcasts a real-time Motion JPEG stream of the robot's camera feed (Figure 4.3a), overlaid with MediaPipe's skeletal landmark detections. The temporal classifier's output is plotted alongside confidence values, allowing developers to see exactly what the robot perceives in real-time.

- **Adaptive Stream Throttling:** The Boston Dynamics Spot's ad-hoc Wi-Fi network is chronically prone to bandwidth limitations and latency spikes [57]. To counteract this degradation, the backend dynamically throttles the video stream resolution and JPEG compression quality based on network queue pressure, ensuring the control signals remain unblocked.

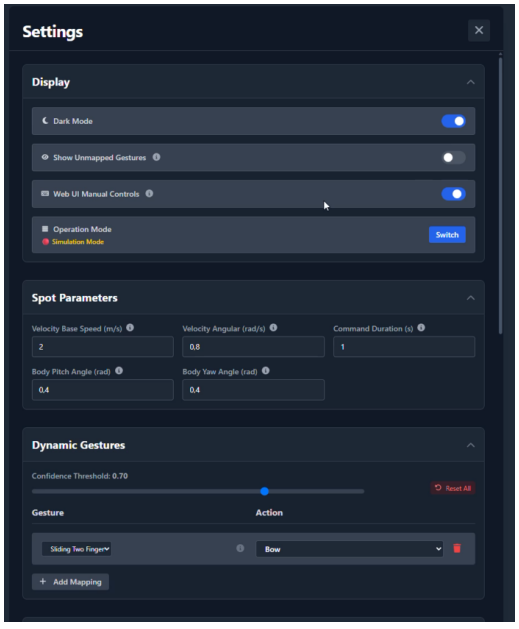
- **Safety & Failsafes:** The UI provides an immediate safety override via a hard E-Stop, triggerable via the spacebar. As a functional fallback, operators can also take manual control of the chassis using standard WASD keyboard controls.

- **Simulation Mode:** A dedicated toggle (Figure 4.3b) allows the system to run the full recognition pipeline while bypassing the physical motor APIs. Instead of commanding the robot, the intended actions and their execution timers are printed on the display, enabling safe validation of gesture combinations.

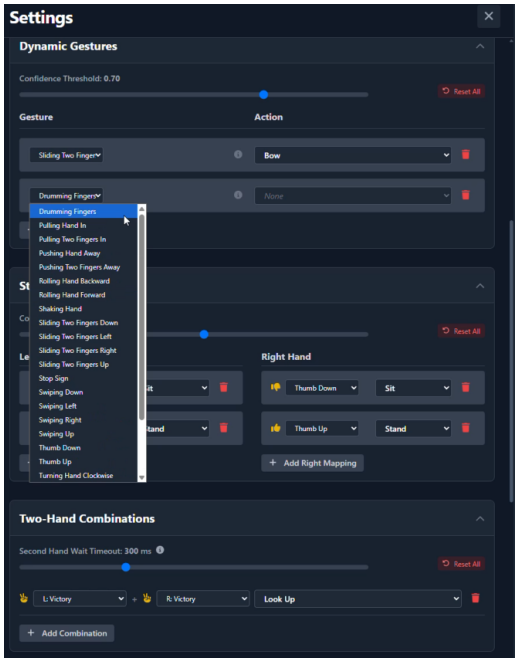
- **Process Control & Dynamic Configuration:** The Python execution engine can be remotely started and stopped directly from the dashboard. Furthermore, operators can hot-swap gesture-to-action mappings (Figure 4.3c), adjust confidence thresholds, tune temporal series timeouts, and modify Spot parameters, such as base walking speeds or body height offsets, without needing to restart the backend.



(a) Live Stream Viewport



(b) System Configuration



(c) Action Mapping

Figure 4.3: Web dashboard interface for system monitoring and configuration. (a) The primary viewport displaying the continuous camera stream with overlaid spatial landmarks. (b) The system configuration panel containing the toggle for Simulation Mode. (c) The dynamic mapping configuration, allowing operators to assign detected gestures to specific robotic actions.

4.9 Deployment & System Orchestration

The complete pipeline, from computer vision inference to the Flask backend, is fully containerized using Docker to ensure reproducibility across disparate environments. A video demonstration of the fully deployed system, successfully mapping continuous human gestures to physical robotic locomotion, is available in the project repository at `https://github.com/Bennit99/robot-gesture-control`. A primary `start.sh` entrypoint script orchestrates the Docker Compose build and run cycles. Internally, Python dependency management is handled via `uv`, a efficient package installer that ensures deterministic environments. Crucially, the architecture maintains a strict separation between offline model training and live deployment. The Docker images are heavily optimized to minimize their footprint by utilizing `uv` dependency groups to strictly exclude resource-intensive training libraries. Furthermore, the `Dockerfile` selectively copies only the specific core modules required for inference (excluding large-scale offline datasets, training scripts, and experimental result logs).
To support both the target robotic hardware and local developer workstations, the script dynamically parses the host CPU architecture (`arm64` vs. `amd64`) to apply specific hardware presets:

- **Jetson Preset (`arm64`):** Binds the container directly to the NVIDIA GPU drivers via the NVIDIA Container Toolkit and passes through the Universal Serial Bus (USB) webcam hardware (`/dev/video0`). Upon first boot, it executes an internal script to natively compile the PyTorch models into TensorRT `.engine` files tailored specifically for the Jetson's GPU architecture.

- **Windows Preset (`amd64`):** Because Docker Desktop on Windows cannot natively pass through local USB webcams, the orchestration script automatically detects attached cameras and spins up an `ffmpeg` background process. This bridge streams the raw webcam feed over User Datagram Protocol to the container, allowing developers to test and modify the gesture system locally without requiring the physical robotic hardware.

Beyond environment virtualization, deploying to headless edge devices often introduces workflow friction. To achieve a zero-touch update process, a dedicated `setup_gesture.sh` routine acts as an automated continuous deployment pipeline directly on the Jetson. When executed, the script verifies network connectivity, dynamically mounts the external solid-state drive containing the repository, and pulls the latest source code from Git. Crucially, it parses the Git output to determine if new commits were fetched. If the codebase was updated, it automatically triggers a full Docker rebuild before launching the container. Otherwise, it executes a rapid restart. This completely abstracts the deployment complexity from the end user.

4.10 User Study Design

To empirically validate the real-world efficacy of the proposed interface, a qualitative user study was conducted. The primary objectives of this study were threefold. First, it aimed to determine whether a vision-based hand gesture control scheme can provide a positive user experience. Second, it compared the five core gesture types against one another to identify the most favorable interaction mechanics. Third, it benchmarked the overall gesture system against Boston Dynamics' standard tablet controller.

4.10.1 Experimental Setup & Task Design

The study recruited a total of 25 participants. Of this cohort, eleven individuals were affiliated with the robotics laboratory or were actively pursuing academic studies in informatics. Consequently, the participant pool does not represent a randomized cross-section of the general public. This demographic composition introduces a distinct selection bias toward high technical literacy and familiarity with complex computational systems. While this specialized sample must be acknowledged, it provides highly relevant insights for the intended use case. Industrial teleoperation systems are typically operated by specialized personnel with a baseline understanding of technical interfaces. The elevated technical understanding of the informatics and robotics cohort closely parallels the cognitive profile of these target end-users. Therefore, evaluating the interface with this specific demographic yields ecologically valid data regarding how skilled operators adapt to vision-based control paradigms. Testing the complete combinatorics of the gesture vocabulary is practically infeasible, as it comprises 6 static classes, 36 two-hand permutations, 36 slider combinations, 25 dynamic classes, and theoretically unlimited stacked sequences. Consequently, one or two representative gestures were selected for each of the five core categories. Arbitrary robot actions were mapped to these representatives to simulate real-world commands:

- **Static:** "Thumbs Up" to stand up, "Thumbs Down" to sit.
- **Two-Hand:** "Double Peace" to execute a 360° turn.
- **Dynamic:** "Sliding Two Fingers Left" to trigger the yoga routine.
- **Slider:** "Flat Hand" combined with "Pointing Up" (index finger) to continuously adjust the chassis height.
- **Stacked:** A sequence of *(Pointing Up → Peace → Pointing Up)* to turn left, and *(Pointing Up → Metal → Pointing Up)* to turn right.

Testing Protocol: The study was conducted in two phases. In the first phase, the representative gestures were demonstrated to the participants one by one. The participants were then instructed to test the gestures themselves, with explicit encouragement to limit-test the system (e.g., varying execution speed, angles, and distances).

In the second phase, a contextual task was introduced to directly compare gesture control against the traditional tablet controller. To simulate an industrial or warehouse scenario, participants were tasked with manually manipulating a large cardboard box that required both hands to hold. While occupied with this physical task, they were instructed to command the robot to execute a 360° turn. They performed this task twice: once using the "Double Peace" gesture, and once using the standard tablet interface, which required them to walk over to pick up the tablet from a nearby table to issue the command.

4.10.2 Survey Methodology & Questionnaire Rationale

Upon completing both runs, participants were administered a qualitative survey utilizing a standard Likert scale [29]. Human attitudes and experiences are rarely binary. If users were simply asked whether a gesture was "easy" or "hard," a binary response lacks the granularity required for meaningful evaluation. Likert scales are widely employed in research involving human opinions because they capture the magnitude and intensity of an attitude across a spectrum. By transforming qualitative opinions, such as perceived effort and naturalness, into quantitative ordinal data with scores from one to five, the responses can be plotted, averaged, and statistically analyzed to mathematically evaluate the mechanics. Specifically, a five-point scale ranging from *Strongly Disagree* to *Strongly Agree* was chosen over an even-point forced-choice scale to explicitly provide a neutral midpoint. This prevents response bias by allowing participants to accurately reflect indecision or neutrality regarding specific gesture mechanics [12]. The core of the questionnaire evaluated each of the five representative gesture types, including Static, Two-Hand, Dynamic, Slider, and Stacked, individually across three distinct categories to isolate the human factors involved in robotic teleoperation:

- **Precision:** Assessed via the statement *"I felt I had precise control over the robot using this gesture."* This metric isolates the perceived latency of the inference pipeline and the effectiveness of the temporal smoothing filters.
- **Naturalness & Intuitiveness:** Evaluated through the statement *"This gesture felt intuitive and natural to perform."* This assesses whether semantic gesture mappings reduce the mental friction of teleoperation compared to traditional joysticks.

- **Physical & Mental Effort:** Evaluated through the statement *"This gesture was mentally and physically straightforward to execute."* Because optical hand tracking requires continuous arm suspension, this identifies the practical limits of prolonged bimanual teleoperation.

Beyond evaluating the core gesture classes against these three metrics, participants were asked targeted questions regarding specific system mechanics. These included evaluating whether the 4-second time limit for stacked gestures felt rushed, and whether the index finger implementation of the slider felt precise. Finally, users were asked to rank the gesture types by overall preference and explicitly state whether they found the gesture control superior to the standard tablet controller for the given task.
By explicitly separating these metrics, the study aims to identify not only whether the gesture system is a viable alternative to the tablet, but specifically in which operational domains it excels.

4.11 Summary

In summary, this chapter detailed the comprehensive experimental framework designed to empirically validate the proposed gesture recognition architecture. It defined the specific hardware constraints of the target edge deployment environment, established the benchmark configurations for evaluating spatial feature extraction efficiency, and outlined the detailed scaling methodology used to optimize the temporal classifier. Furthermore, the chapter formalized the design and execution parameters of the real-world user study, ensuring that the qualitative evaluation of the system's ergonomics and usability is grounded in statistically sound metrics.

5 Results

This chapter presents the comprehensive evaluation of the proposed gesture control system. The analysis begins by benchmarking the spatial feature extraction efficiency of the MediaPipe framework against YOLO-based alternatives. Subsequently, the temporal classification accuracy of the GRU is evaluated across the full Jester dataset. The technical performance of the completely integrated pipeline is then comprehensively profiled, detailing the system's real-time latency, throughput, and thermal power draw on the target edge hardware. Finally, the chapter presents the empirical findings from a formal user study, quantifying HRI preferences and the perceived ergonomics of the vision-based interface.

5.1 Spatial Feature Extraction Benchmark

This section evaluates the performance and efficiency of the spatial feature extraction pipeline. The analysis is divided into two primary parts. First, the detection rate and computational footprint of the MediaPipe framework are benchmarked against a YOLO-based baseline to explicitly justify the architectural selection. Second, the spatial reliability of the chosen MediaPipe model is deeply analyzed across the entire dataset to identify specific failure modes, temporal detection inconsistencies, and class-specific anomalies.

5.1.1 YOLO vs MediaPipe Detection Rates

Across the approximately 5.1 million frames of the filtered Jester dataset, both architectures exhibited highly similar detection rates. Specifically, MediaPipe successfully detected hands in 50.1% of the frames, while the YOLO11n-pose baseline detected hands in 49.5% of the frames. It should be explicitly noted that these statistics exclude the *No gesture* class. The *No gesture* class naturally features zero hands in the camera frame for its entire duration. Retaining these $4,811$ video sequences would artificially drag down the global average detection rate and heavily skew the baseline metrics for the actual hand gestures being benchmarked.
Furthermore, it is important to explain this seemingly low absolute percentage. The raw videos in the Jester dataset are not tightly trimmed. Consequently, a significant portion of the frames at the beginning and end of each video contain no visible hands, substantially lowering the theoretical maximum detection rate. Therefore,

these metrics reflect the natural sparsity of the dataset, as illustrated in Figure 5.1, rather than a failure of the extraction models.

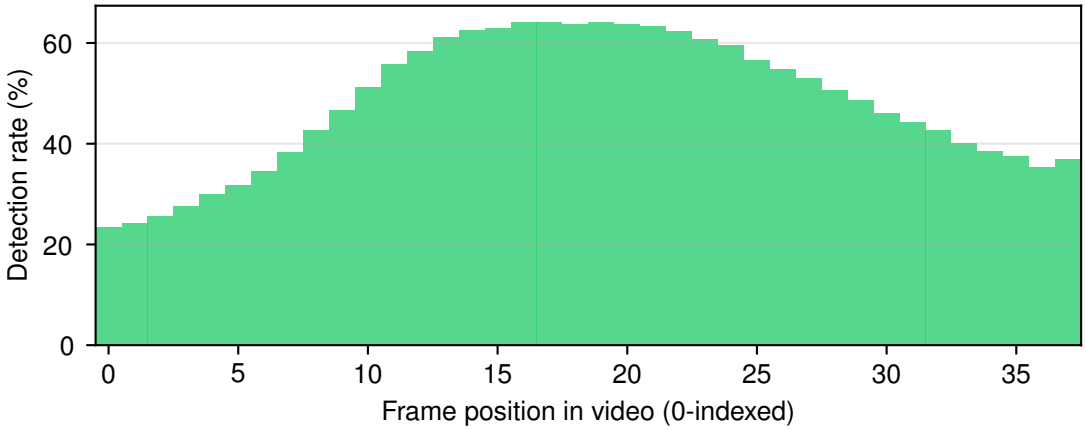


Figure 5.1: Hand detection rate across normalized frame positions in the Jester dataset (N = 143,281 videos). The data illustrates that instances of non-detection occur at the immediate beginning and end of the loosely trimmed video sequences, rather than during the core gesture execution.

MediaPipe proved quantitatively superior by identifying hands in approximately 1.2% more frames relative to the baseline. However, this automated metric does not account for false positives or spatial precision, as the Jester dataset lacks ground-truth landmark annotations. To facilitate an objective spatial comparison, a manual qualitative evaluation was conducted on a randomized sample of 1,000 discordant frames. This sample isolates only the "hard examples" of the dataset, specifically instances where the two models actively disagreed on the presence or location of a hand. Because the architectures produced identical outputs for the vast majority of the 5.1 million frames, analyzing only these outliers deliberately highlights edge-case failure modes and should not be misinterpreted as suboptimal overall performance. To formalize these findings, the discordant frames were categorized based on the nature of the disagreement, as detailed in Table 5.1.

Disagreement Category	**MediaPipe Correct**	**YOLO Correct**	**Both Failed**	**Total**
MediaPipe detected more hands	325	78	41	444
YOLO detected more hands	149	321	70	540
Different spatial locations	16	0	0	16
Total	**490**	**399**	**111**	**1,000**

Table 5.1: Results of a manual blind evaluation of 1,000 discordant frames. The data reveals that the model detecting the higher number of hands is predominantly correct, indicating that Non-detections, or missed detections, are the primary failure mode for both architectures.

A blind visual inspection of this dataset reveals that MediaPipe correctly identified the true hand configuration in 490 frames, whereas YOLO succeeded in only 399 instances. Both models failed completely in the remaining 111 frames. A binomial test confirmed that this disparity in overall accuracy is highly statistically significant ($p < 0.01$).

Crucially, the categorical breakdown in Table 5.1 yields two critical insights into the behavior of the spatial extractors. First, it demonstrates that false negatives, missing hands due to motion blur or occlusion, are a substantially more common failure mode than false positives, halluncinating non-existent hands. If hallucinations were the dominant issue, the model detecting *fewer* hands would consistently be correct. Instead, the model detecting *more* hands was correct in nearly 66% of cases (646 out of 984). However, analyzing the architectures individually reveals a significant divergence. When MediaPipe detected hands that YOLO missed, MediaPipe was correct 73% of the time (325 out of 444), demonstrating a strong resistance to false positives. Conversely, when YOLO detected hands that MediaPipe missed, YOLO was correct only 59% of the time (321 out of 540). This proves that while both models primarily suffer from false negatives, YOLO is nearly twice as likely to hallucinate a non-existent hand (149 false positive instances) compared to MediaPipe (78 instances). Second, in the rare instances where both models detected the same number of hands but placed them in entirely different spatial locations, MediaPipe's bounding box regression was universally correct, winning 16 to 0.

Beyond this proven spatial superiority, MediaPipe also offers a critical advantage in computational efficiency. As detailed extensively in the subsequent spatial profiling analysis, empirical testing reveals that the YOLO11n-pose baseline demands 7.9 GFLOPs per frame, whereas the MediaPipe pipeline requires only 1.34 GFLOPs. This 5.9× reduction in computational cost occurs despite MediaPipe containing nearly twice the parameter count (4.8M vs 2.6M). This counter-intuitive disparity exists because MediaPipe's decoupled ROI architecture strictly minimizes the spatial area processed by dense convolutions. In autonomous robotics, active computational operations (FLOPs) directly dictate power draw and battery drain, acting as the primary operational bottleneck. Conversely, a static parameter count merely dictates memory utilization. Storing 4.8M parameters at 16-bit precision requires less than 10 Megabytes of memory, a strictly trivial footprint for the Jetson AGX Orin's 64 GBs of unified memory.

Due to this computational disparity and comparable detection rates, MediaPipe was decisively selected as the spatial feature extractor.

5.1.2 MediaPipe Landmark Detection Analysis

To further analyze the reliability of the spatial extraction pipeline, the detection rate was evaluated on a per-video basis. This evaluation reveals a negatively skewed distribution featuring absolute boundary spikes, as illustrated in Figure 5.2. The data exhibits two extreme boundary peaks at 0% and 100% detection rates. The extreme lower bound of 0% represents a total failure of the MediaPipe palm detector to identify any hands throughout the entire video sequence. Manual qualitative in-

spection of these specific outlier sequences reveals severe environmental constraints. Many of these videos were recorded in rooms illuminated solely by the glow of a computer monitor. Consequently, the inexpensive webcams utilized by the crowd-sourced participants automatically maximized exposure times to gather sufficient light. This artificially high shutter speed introduces extreme motion blur during dynamic hand gestures. The resulting blur destroys the structural contours required by the MediaPipe palm detection model.

To mitigate this hardware-induced failure mode in the deployed robotic system, three strategies can be implemented. First, the most trivial solution is to ensure the operational environment maintains adequate ambient lighting, avoiding the camera's need to overexpose the sensor. Second, utilizing a camera equipped with a larger lens aperture significantly increases the volume of light striking the sensor per frame. The external webcam utilized for live teleoperation in this thesis possessed a sufficiently large aperture to passively prohibit motion blur even in moderately low-light environments. Finally, if low-light operation on constrained hardware is unavoidable, the camera's exposure settings must be manually overridden. The shutter speed should be clamped to a maximum duration, such as 1/100 of a second, to physically freeze rapid hand movements. To compensate for the resulting underexposure, the camera's ISO sensitivity must be artificially increased. While a higher ISO introduces digital noise and grain into the image stream, modern neural networks are highly resilient to additive spatial noise, yet fail completely when structural contours are erased by motion blur [23, 13]. Furthermore, this failure peak captures exceptionally rapid or ambiguous gestures that remain difficult even for human annotators to interpret.

Conversely, the second peak at 100% in Figure 5.2 represents optimal capture conditions. In these instances, participants positioned their hand within the camera frame before recording began and maintained visibility until the sequence concluded. Between these two absolute extremes, the distribution forms a negatively skewed curve concentrated around an average detection rate of 70%. This 70% concentration mathematically corresponds to approximately half a second of empty frames before and after the core gesture execution. This aligns directly with the temporal padding observed in the positional detection data shown in Figure 5.1.

Analysis of the class-separated data, as illustrated in Figure 5.3, reveals distinct behavioral patterns across the gesture categories. As expected, the baseline *No gesture* class exhibits a 96% non-detection rate, confirming its structural definition. Additionally, MediaPipe identifies two hands in 2.6% of all processed frames. Given that the 25 core gesture classes are designed strictly as single-hand operations, this secondary detection is generally incidental. This behavior predominantly occurs within the *Doing other things* class, which exhibits a higher two-hand detection rate of 7%. This elevated rate is a natural consequence of participants performing unstructured, baseline actions that frequently involve bimanual coordination.

Beyond the background categories, highly dynamic movements such as swiping and sliding exhibit systematically lower single-hand detection rates. This reduction is driven by three primary factors. First, broad spatial movements frequently cause the participant's hand to exit the camera frame entirely. Second, rapid execution

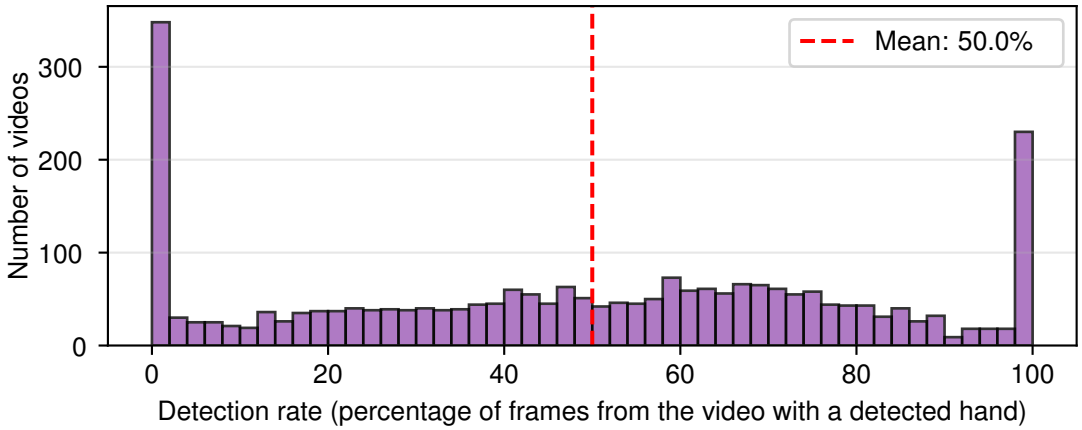


Figure 5.2: Distribution of per-video hand detection rates across the dataset ($N = 143{,}281$ videos). The histogram demonstrates that most video sequences maintain a high detection consistency, with the overall average heavily skewed by a minority of outliers.

introduces severe motion blur that degrades the structural contours required for successful landmark extraction. Finally, dynamic actions are often completed much faster than the total duration of the video clip, leaving significant temporal padding where no hand is visible. A notable exception to this dynamic trend is the *Pushing Hand Away* gesture, which maintains a higher detection rate. This occurs because the action is performed along the camera's depth axis, ensuring the hand remains centered within the frame before being safely retracted.

Conversely, gestures characterized by minimal spatial displacement yield significantly higher detection rates. Because these localized actions can often be held continuously, similar to an open-palm *Stop Sign* gesture, they tend to fill more of the video duration, are less likely to exit the frame during the performance, and significantly reduce motion blur. The *Thumb Up* and *Thumb Down* classes serve as notable exceptions, exhibiting lower tracking performance than other static poses despite being held continuously. While this indicates a weakness in the spatial extractor for these specific shapes, it does not represent a critical system failure. The overarching control architecture incorporates an independent, dedicated static pose classifier specifically designed to reliably identify these configurations regardless of continuous landmark tracking performance.

To understand the temporal nature of these tracking failures, we analyzed the length of consecutive missing-frame sequences, as presented in Figure 5.4. The resulting distribution reveals two distinct modes of failure. The most frequent occurrences are brief mostly single-frame isolated tracking losses, forming a sharp peak at the extreme lower end of the histogram. These fractional-second failures typically result from momentary motion blur, sensor noise, or minor algorithmic jitter.

A secondary, broader peak emerges around eight consecutive missing frames. This cluster corresponds directly to the temporal padding at the start and end of the video sequences, where the participant's hand has either not yet entered the camera view or has already been retracted. Beyond this secondary peak, the frequency of missing runs naturally decreases until approximately 30 frames. Any missing sequences

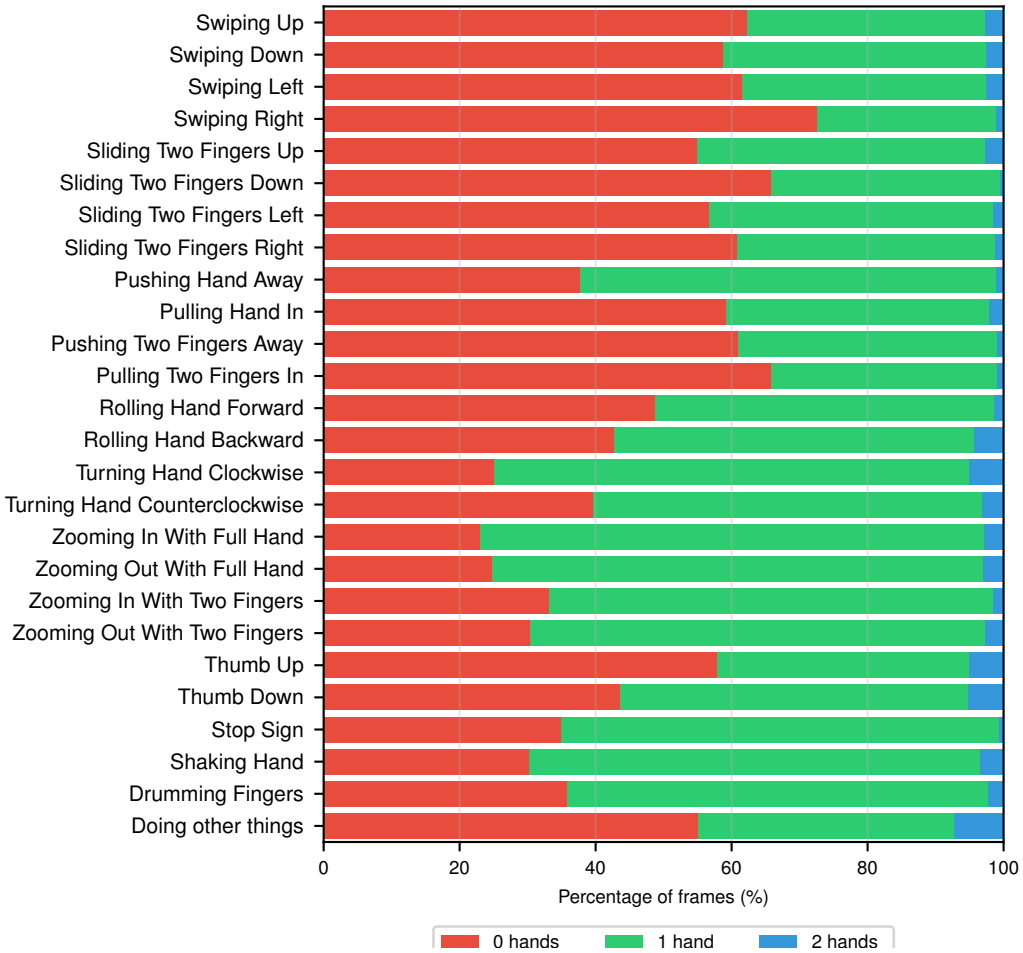


Figure 5.3: Distribution of detected hands per frame, separated by gesture class. The Doing other things class exhibits a disproportionately high rate of two-hand detections compared to the 25 core single-hand gestures.

extending beyond 30 frames generally indicate a catastrophic tracking failure. These extreme outliers represent sequences where MediaPipe failed to identify the hand for nearly the entire duration of the recording.

5.2 Temporal Classification

Before training the temporal classification models, the dataset was strictly filtered to remove videos where MediaPipe failed to consistently extract spatial landmarks. When training recurrent or temporal architectures on landmark data with exceptionally few detected frames, the model lacks sufficient context to identify the spatio-temporal pattern. This data deficiency essentially forces the model to guess the gesture class, a failure mode clearly visualized in Figure 5.5. Avoiding this guessing behavior is critical for reliable real-world inference on the robotic platform.

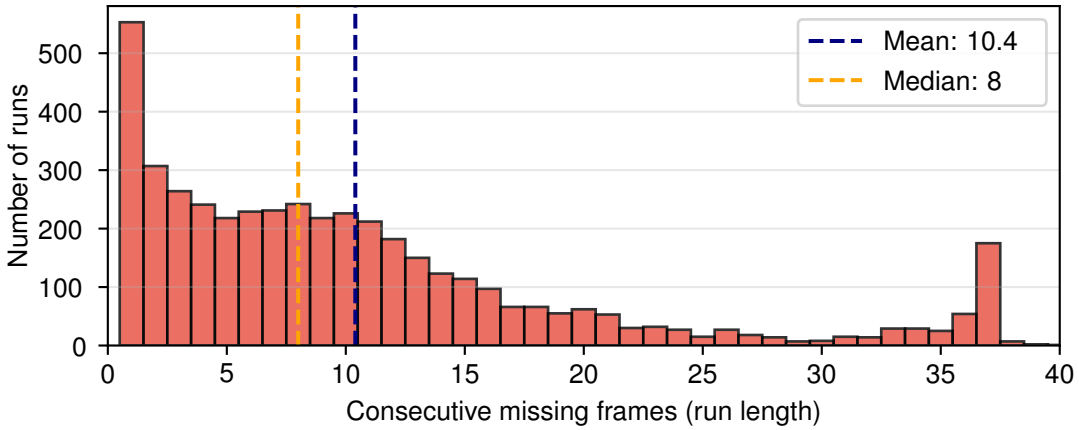


Figure 5.4: Distribution of consecutive missing-frame run lengths ($N = 245{,}920$ missing runs). The histogram highlights single-frame jitter at the lower extreme and padding-related absences peaking around eight frames.

As established in the previous section, these detection failures typically occur during extremely rapid gestures causing severe motion blur or in excessively dark environments. The specific threshold of nine successfully tracked frames was derived empirically from two key observations. First, as shown in Figure 5.4, the frequency of consecutive missing frames drops near zero at 30 frames but unexpectedly rises again at 31 frames and beyond. In a fixed 40-frame sequence, missing 31 or more frames implies that at most nine frames were successfully tracked. This late spike indicates a catastrophic tracking failure where MediaPipe completely loses the hand. Second, Figure 5.5 visually supports this decision. The ratio of correct to incorrect temporal classifications significantly improves starting at the 22% detection mark, which corresponds to nine out of 40 frames.

Applying this nine-frame minimum threshold across the dataset, excluding the *No gesture* class, resulted in 34,532 dropped videos. This equates to a 24.10% rejection rate and yields a highly reliable core dataset for training.

This thresholding strategy is driven by practical requirements for robotic teleoperation. Training a temporal classifier on severely corrupted or flickering landmark data introduces unnecessary noise and degrades overall model stability. In a real-world deployment, acting upon unstable spatial extractions poses a significant safety risk. By explicitly filtering out sequences where MediaPipe fails to maintain a reliable track, the model is optimized for deterministic execution under nominal conditions. While this filtering strategy limits direct comparability with end-to-end models evaluated on the unpruned dataset, it aligns the evaluation metric with the operational reality of the proposed decoupled architecture. Furthermore, the enforced sequence length ensures that brief, inadvertent movements cannot trigger a command.

Class-Level GRU Evaluation

Beyond the aggregate metrics, analyzing the performance of the GRU model on a per-class basis reveals both the capabilities and the inherent challenges of monoc-

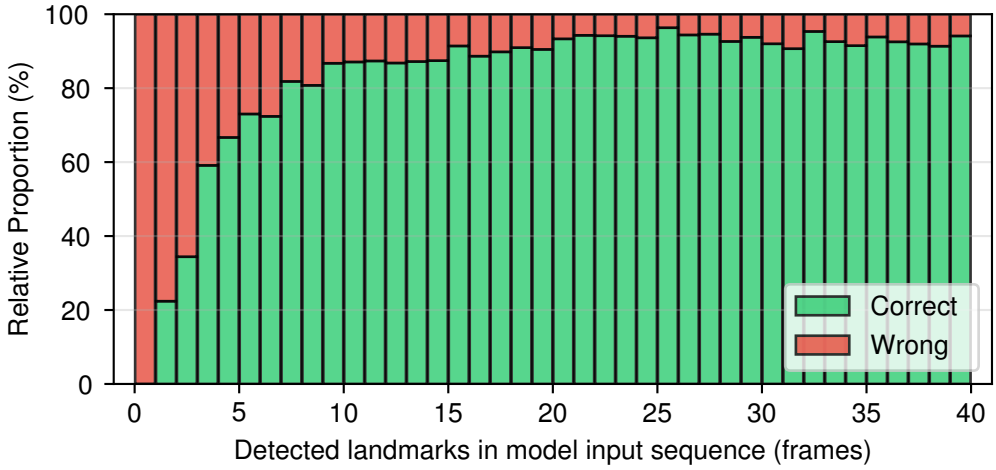


Figure 5.5: Correlation between MediaPipe detection rate and final GRU-based temporal classification accuracy, excluding the No gesture class ($N = 14{,}254$ validation videos). Notably, the prominent spike at a 100% detection rate observed in the earlier dataset statistics in Figure 5.2 is diminished here. This occurs because the classification pipeline pads all sequences to a fixed length of 40 frames. Consequently, naturally short videos that previously had perfect detection rates now contain empty padding frames, reducing their overall detection percentage. Videos with extremely low detection rates inherently lack the temporal context required for correct classification.

ular gesture recognition. The strongly pronounced diagonal across the confusion matrix visually confirms that the model correctly classifies the vast majority of gestures, achieving an overall accuracy of 91.6% across all 27 categories. However, as illustrated by the subset confusion matrix in Figure 5.6, performance still varies significantly depending on the structural complexity of the gesture.

The most challenging categories were *Turning Hand Clockwise* and *Turning Hand Counterclockwise*. These rotational gestures are notoriously difficult to classify at a capture rate of 12 frames per second (FPS) because they are executed in a single spatial location. When the temporal resolution is low, the model frequently confuses the two turning directions with one another, an effect that is prominently visible in the confusion matrix. Conversely, directional gestures such as *Swiping Left* and *Swiping Right* are easily distinguished, as the spatial location of the hand changes distinctly over time.

Gestures with highly distinct, localized motions or rigid static poses yielded exceptional accuracy. The *Drumming Fingers* and *Thumb Down* gestures were the best performing classes. These gestures maintain consistent landmark visibility and do not suffer from the severe self-occlusion issues present in complex rotational movements.

The *No gesture* baseline class was also easily detected. This high accuracy stems from the fact that no hand should be present in the frame during these sequences, making them inherently distinct from all active gesture classes. However, misclassifications occasionally occur because 2.2% of the frames in the dataset contain hand landmarks when none should be detected. Visual inspection reveals that these

instances are false positive detections by MediaPipe, rather than participants mistakenly introducing their hands into the frame. Finally, the *Doing other things* class contains arbitrary hand movements that do not belong to any specific gesture category. This class ensures that the model learns to differentiate between intentional command gestures and random motions. Consequently, if a gesture is executed poorly or if MediaPipe extracts noisy landmarks, the model defaults to predicting this class.

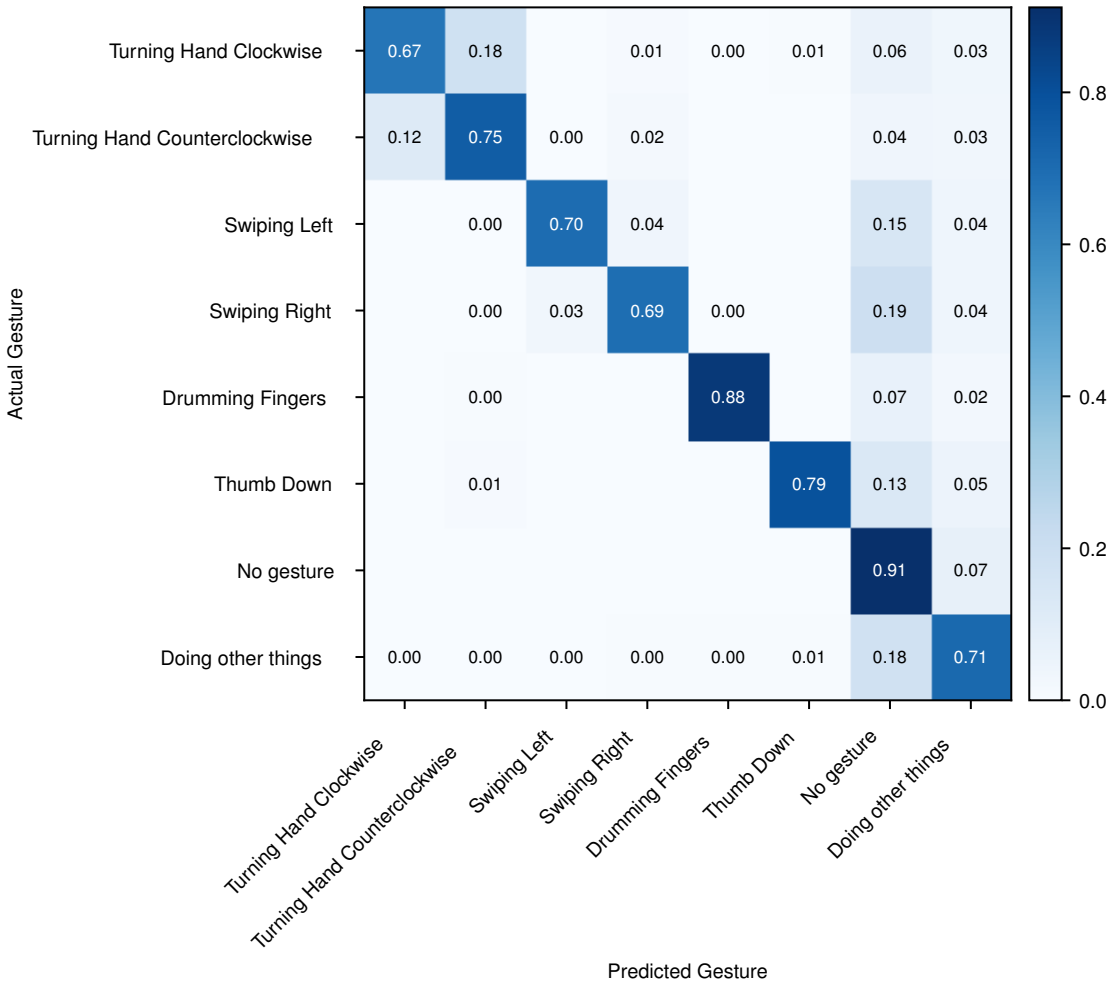


Figure 5.6: Confusion matrix of a representative 8-class subset from the GRU model evaluation. The full 27-class matrix is provided in Figure A.1 in the appendix.

Analysis of Validation versus Training Accuracy

During the evaluation of the Transformer model, validation accuracy consistently exceeds training accuracy. This phenomenon is a hallmark of a robustly regularized network and is driven by three primary architectural mechanisms [56]:

- **Substantial Dropout Regularization:** The Transformer relies heavily on Dropout to prevent overfitting. During the training phase, dropout randomly

zeroes out 10% to 30% of the connections within the multi-head attention blocks, feed-forward networks, and positional encodings. This effectively limits the network capacity. During validation, dropout is disabled, granting the model access to its full computational capacity and immediately yielding superior performance.

- **Epoch Averaging versus End-of-Epoch Evaluation:** Training accuracy is computed as a running average across the epoch, combining predictions from early batches with later batches. Validation accuracy is evaluated strictly at the end of the epoch using the finalized optimal weight parameters.
- **Data Augmentation Disparity:** If data augmentation or noise imputation is utilized during training, the network is forced to learn on artificially harder data. The validation set is evaluated on noise-free, ground-truth samples, leading to a naturally higher accuracy.

Architectural Pareto Frontier and Model Selection

To systematically determine the most efficient architecture for real-time edge deployment, the models were evaluated across a range of computational budgets. Figure 5.7 illustrates the resulting Pareto frontiers, mapping validation accuracy against the computational cost per inference step on a logarithmic scale.

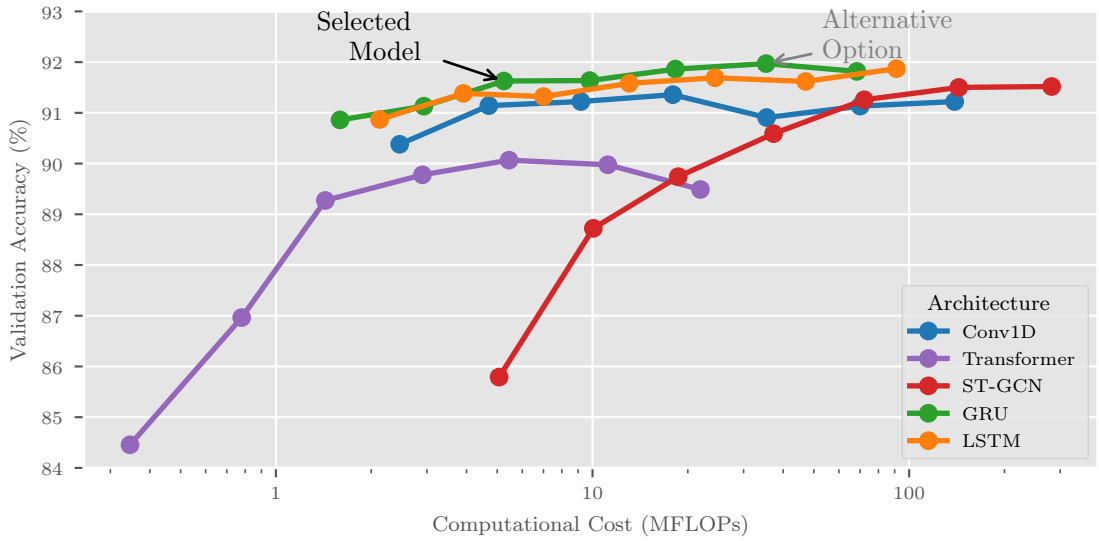


Figure 5.7: Validation accuracy across different architectural scales, plotted against the computational cost per inference step (MFLOPs) on a logarithmic x-axis. The GRU architecture establishes the optimal Pareto frontier. The selected model configuration strikes the ideal balance between accuracy and computational efficiency for real-time robotic control.

The analysis reveals that the GRU consistently establishes the optimal Pareto frontier across all computational scales, outperforming all other evaluated architectures: the LSTM, spatial-temporal convolutions (Conv1D), ST-GCN, and the Transformer. Notably, highly complex architectures such as the Transformer fail to reach the peak

accuracy of the GRU across any tested scale. At lower computational budgets, their performance is significantly inferior, indicating that self-attention mechanisms require substantial parameter counts to function effectively. Conversely, as their capacity scales up to higher computational budgets, they rapidly overfit the dataset. This is evidenced by a distinct drop in validation accuracy at the largest model scales. Consequently, strong regularization is necessary on bigger transformer model architectures to achieve higher accuracies.
Based on this quantitative evaluation, a specific GRU configuration was selected for the final deployment. The selected model (highlighted in Figure 5.7) achieves a validation accuracy of 91.63% while consuming a mere 5.25 MFLOPs per inference step. While the peak GRU configuration achieves a slightly higher accuracy of 91.97%, it demands 35.33 MFLOPs per step. In the context of battery-powered edge robotics, this marginal 0.34 percentage point improvement in accuracy does not justify the 6.7× increase in computational cost. By prioritizing extreme computational efficiency, the selected 5.25 MFLOP model ensures minimal thermal throttling and power draw while maintaining exceptional gesture recognition capabilities. However, it must be noted that for unconstrained deployment environments, such as server-side inference the larger peak-accuracy GRU configuration remains a highly viable alternative.

Spatial Extraction Efficiency: YOLO versus MediaPipe

To accurately quantify the computational efficiency of the spatial extraction pipeline, empirical FLOP calculations were performed directly on the model artifacts. The YOLO11n-pose-hands model was profiled natively using the PyTorch Ultralytics profiler framework, yielding an inference cost of 7.9 GFLOPs per frame. In contrast, the MediaPipe pipeline utilizes heavily optimized TensorFlow Lite kernels which do not natively expose parameter profiles in Python. To circumvent this, the `hand_landmarker.task` archive was extracted and its underlying `.tflite` flatbuffer schemas were parsed using the `tflite-flops` profiling tool. The empirical testing revealed that the MediaPipe hand detector demands 674.7 MFLOPs, while the subsequent hand landmark regressor demands 661.6 MFLOPs. Consequently, the complete MediaPipe spatial pipeline consumes a maximum of 1.34 GFLOPs per frame. It is important to note that many edge-computing publications report Multiply-accumulate operations (MACs) rather than raw FLOPs. Since one MAC equates to two FLOPs, representing one multiplication and one addition, MediaPipe's 1.34 GFLOPs is mathematically equivalent to approximately 670 MMACs. This validates that the MediaPipe pipeline performs the necessary spatial tracking at roughly one-sixth the computational cost of the YOLO alternative, confirming its suitability for edge deployment.

5.2.1 Impact of Landmark Interpolation

To systematically evaluate the efficacy of the linear landmark imputation strategy detailed in Section 3.3, an ablation study was conducted across the temporal clas-

sification dataset. The theoretical objective of the interpolation was to artificially bridge transient spatial tracking failures caused by motion blur, thereby providing the GRU with a continuous and fluid sequence of kinematic features.
To quantify this, eight distinct temporal classifiers were trained from scratch. For each model, the dataset preprocessing pipeline was configured with a different maximum interpolation gap hyperparameter, systematically varied from zero (disabling imputation entirely) to seven consecutive frames. Each independently trained model was then evaluated on the correspondingly preprocessed validation set. As illustrated in Figure 5.8, the empirical results strictly contradicted the initial hypothesis. Disabling the interpolation entirely (a maximum gap of zero frames) yielded the highest baseline validation accuracy of 91.63%. Incremental increases to the maximum allowed interpolation gap resulted in a monotonic degradation of model performance, culminating in a drop to 91.01% when permitting mathematical imputation across seven consecutive missing frames.

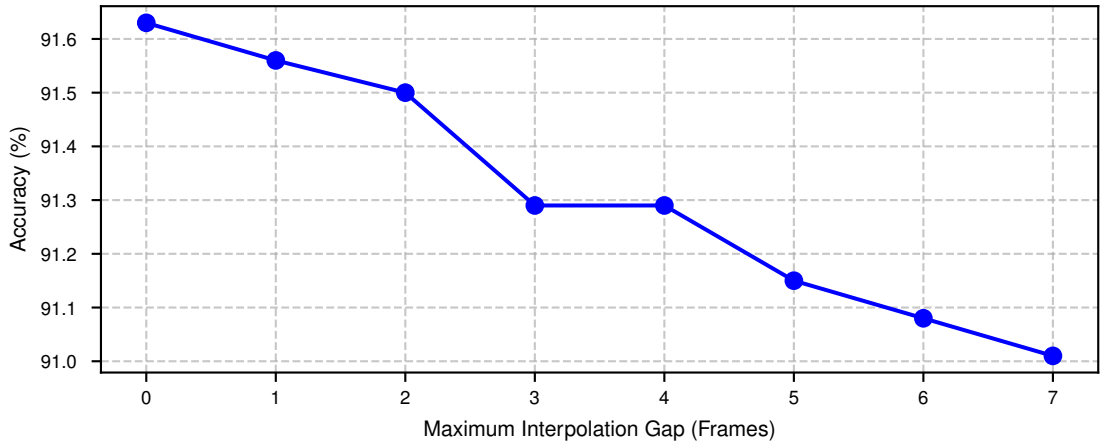


Figure 5.8: Ablation study demonstrating the monotonic degradation of temporal classification accuracy as the linear landmark imputation gap tolerance is increased.

This continuous degradation suggests that mathematically bridging motion blur gaps introduces rigid, artificial kinematic artifacts into the data stream. Rather than aiding the temporal network, these linear approximations confuse the robust spatio-temporal representations learned by the model. As demonstrated by Che et al. [8] in their foundational work on multivariate time series, GRUs possess a strong native capacity to model missing data directly when represented as zero-vectors. The internal gating mechanisms of the network inherently learn to act as an optimal, non-linear internal interpolator, effectively bypassing the missing steps without requiring explicit mathematical imputation. Consequently, the linear landmark interpolation preprocessing step was entirely disabled for the final deployment architecture to preserve optimal inference accuracy.

5.2.2 Comparison with State-of-the-Art

To explain the performance of the proposed architecture, it is compared against existing state-of-the-art models from the literature, specifically focusing on the trade-

off between computational efficiency and accuracy. Table 5.2 summarizes these metrics for several prominent architectures evaluated on the Jester dataset.

Table 5.2: Comparison of the proposed decoupled architecture against state-of-the-art models. The FLOPs metric represents the total system computational cost per frame. Accuracy comparisons must be interpreted cautiously, as evaluation scopes differ significantly across the literature.

Architecture	**FLOPs**	**Parameters**	**Accuracy**	**Evaluation Scope**
Proposed Architecture	1.34 G	2.0 M	91.6%	Full 27 Classes
Def-ResNeXt3D-101 [69]	59.76 G	52.2 M	96.6%	Full 27 Classes
TSM (MobileNetV2) [30]	2.4 G	3.5 M	94.5%	Full 27 Classes
3D-CNN + LSTM [51]	3.8 G	3.7 M	97.5%	15 Classes

When deploying machine learning models on edge devices for robotic teleoperation, the total computational cost, measured in FLOPs, is the most critical constraint. Active mathematical operations directly dictate power draw, thermal output, and battery drain. For baseline architectures where computational costs were omitted in the original publications, PyTorch replicas were constructed based on the authors' architectural descriptions. The `fvcore` profiling library was subsequently employed to statically analyze these models using standardized input tensors, ensuring a consistent and hardware-agnostic calculation of total operations across all evaluated models. As Table 5.2 demonstrates, the proposed architecture operates at a total system cost of approximately 1.34 GFLOPs. This represents a substantial efficiency gain over monolithic 3D convolutional networks such as the Deformable ResNeXt3D-101 baseline, which requires nearly 60 GFLOPs to process a frame. Even when compared to highly optimized edge alternatives such as the TSM using a MobileNetV2 backbone [30], the proposed decoupled approach requires nearly half the compute. While the total parameter count is listed for completeness, minimizing parameters in this low-megabyte range is of secondary importance. Modern edge hardware, such as the Jetson AGX Orin, possesses sufficient unified memory to store models of this scale without bottlenecking execution.

Furthermore, a direct comparison of the reported accuracies across these architectures is fundamentally problematic because the evaluation scopes are inconsistent. While monolithic models often evaluate on the full 27-class challenge of the Jester dataset, many hybrid and lightweight architectures drastically reduce the problem space to artificially inflate accuracy metrics. For instance, Rehman et al. reported a 97.5% accuracy using their 3D-CNN + LSTM framework, but they explicitly state: "Due to time constraints and memory resources restriction, only 15 out of 27 hand gestures are used in this work" [51]. Similarly, standard MediaPipe gesture pipelines are typically evaluated on minimal subsets comprising five to seven static shapes [18]. Furthermore, the proposed pipeline fundamentally relies on the spatial tracking capabilities of MediaPipe. Video samples where this tracking subsystem failed to extract sufficient hand landmarks were filtered out during training and evaluation. While this selective data cleaning removes heavily occluded or out-of-frame sam-

ples, it intrinsically weakens the direct comparability of the final accuracy against models that process raw pixels without a failure-prone extraction step. Nevertheless, achieving roughly 91.6% accuracy on the complete 27-class dynamic dataset at 1.34 GFLOPs positions the proposed decoupled architecture as an exceptionally pragmatic solution for real-time robotic control.

5.3 Hardware Optimization & System Performance

In addition to classification accuracy, the practical viability of the proposed architecture depends heavily on its computational efficiency. This section evaluates the system's real-time inference latency and resource utilization when deployed on edge hardware, specifically the NVIDIA Jetson AGX Orin.

5.3.1 Model Inference Latency

In order to evaluate the real-time capabilities of the gesture recognition pipeline on edge hardware, the inference latency of the isolated dynamic GRU model was benchmarked on the NVIDIA Jetson AGX Orin. The evaluation compares the native PyTorch execution against the intermediate ONNX Runtime and the fully hardware-optimized NVIDIA TensorRT engine described in Chapter 4. Each framework was subjected to 1000 continuous inference iterations following an initial warmup phase.

Table 5.3: Inference latency benchmark of the GRU model across different runtime frameworks on the Jetson AGX Orin.

Framework	**Mean (ms)**	**Min (ms)**	**Max (ms)**	**Speedup**
PyTorch (CPU)	27.65	26.94	99.64	1.0×
TensorRT (GPU)	1.63	1.61	1.65	17.0×
ONNX Runtime (CPU)	1.41	1.38	2.05	19.6×

As summarized in Table 5.3, the native PyTorch execution on the CPU establishes a baseline mean latency of 27.65 ms. This relatively high latency bottlenecks the pipeline and consumes a significant portion of the available computational window. By utilizing the highly optimized C++ CPU execution provider of the ONNX Runtime, the latency drops to a remarkably stable median of 1.40 ms, representing a 19.6× speedup without engaging the GPU.
Interestingly, when evaluated under the previously established MAXN performance mode, the compiled TensorRT engine achieves a highly competitive and remarkably stable median latency of 1.63 ms, representing a 17.0× speedup over the PyTorch baseline. The extensive layer fusion and kernel auto-tuning successfully eliminate runtime variance, yielding a maximum latency of merely 1.65 ms. However, despite this hardware optimization, the engine remains slightly slower than the ONNX CPU execution. This behavior is characteristic for lightweight, strictly sequential GRUs.

The persistent overhead of host-to-device memory transfers and sequential kernel launches limits the parallelization advantages of the GPU.
While these benchmarks prove that the isolated GRU model can theoretically execute in just ~ 1.40 ms under maximum system load profiles, doing so is inefficient for real-world deployment. When combining this execution with image loading and MediaPipe spatial feature extraction, the total end-to-end pipeline requires roughly 3 ms at peak clock speeds. However, the system camera operates at 15 FPS, providing new frames roughly every 66 ms. Because a 66 ms delay is entirely imperceptible in terms of system response time for HRI, achieving extreme single-digit millisecond latency at the cost of disproportionate power consumption is unnecessary. Instead, operating the hardware in a efficient dynamic clock scaling mode naturally extends the total end-to-end pipeline latency to approximately 36 ms. This fits within the required 66 ms computational window while drastically reducing the overall power footprint.

5.3.2 Energy Efficiency and Resource Utilization

The decision to deploy the final gesture pipeline entirely on the CPU is based on three converging factors. First, as established in the latency benchmarks, the GRU temporal classifier executes faster natively on the CPU via ONNX than on the GPU. Second, the spatial feature extractor, Google MediaPipe, features robust GPU acceleration for mobile and web environments via OpenGL ES and WebGL, respectively [19], but its Python bindings for Linux lack native support for NVIDIA's CUDA. Consequently, the standard MediaPipe package is incapable of utilizing the Ampere GPU on the Jetson AGX Orin out of the box, forcing the spatial tracking component to run on the CPU. Third, executing the entire pipeline on the CPU allows the GPU to be completely bypassed. This means the GPU can be repurposed to fulfill other computationally demanding and independent robotic tasks, such as running a complex SLAM model, without interfering with the gesture control system.
Because the GPU remains entirely idle and the CPU utilizes dynamic clock scaling to stretch computation across the available 66 ms window, the system achieves exceptional energy efficiency. To quantify this efficiency, a 90-second power profiling test was conducted on the Jetson AGX Orin. The resulting power and CPU utilization metrics are visualized in Figure 5.9 and Figure 5.10, and the test is segmented into three distinct phases:

1. **Phase 1, the First 30s:** The pipeline is active but no hands are present in the frame. The system draws an average of 5.57 W, with the CPU running at 13.5% utilization and an average clock speed of 825 megahertz (MHz).

2. **Phase 2, the Middle 30s:** Full gesture recognition is running on detected hands. The total power draw slightly increases to 6.43 W, with the CPU reaching 22.6% utilization at 931 MHz.

3. **Phase 3, the End 27s:** The pipeline is terminated and the system returns to an idle standby state. The baseline power draw drops to 4.05 W, and the CPU idles at its minimum clock frequency of 729 MHz.

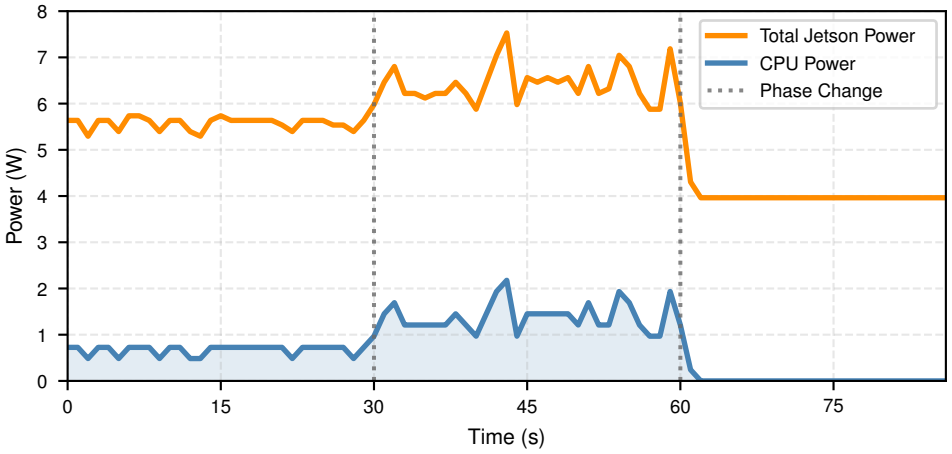


Figure 5.9: Total system power and CPU power draw during the three phases of the 90-second test.

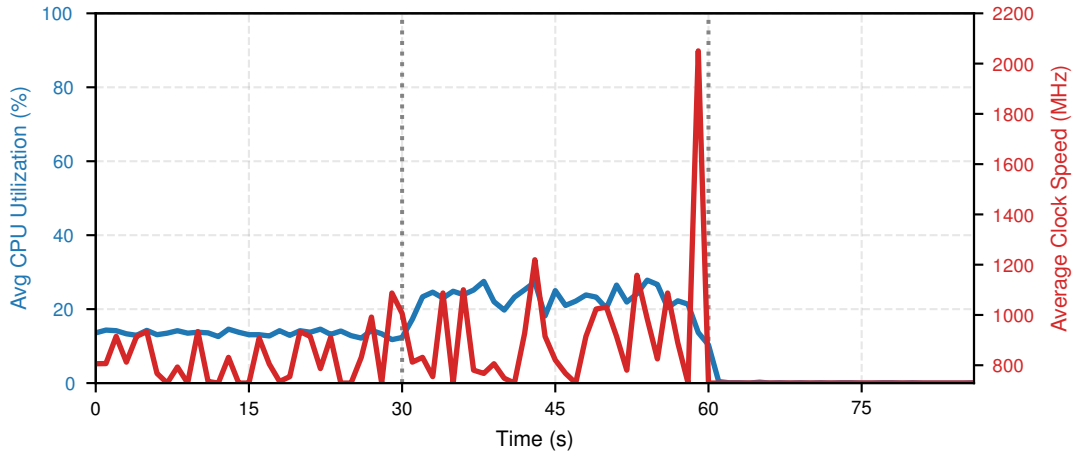


Figure 5.10: Average CPU utilization and clock speed responding dynamically to the pipeline load.

The delta power consumption for executing the entire gesture recognition service is merely ~ 2.38 W, calculated from the 6.43 W full load subtracting the 4.05 W idle baseline as illustrated in Figure 5.9. This is remarkably low. It consumes less power than a standard smartphone charger and is completely negligible compared to the Jetson AGX Orin's maximum configurable power draw of 60 W [46]. As shown in Figure 5.10, the CPU utilizes 22.6% of its capacity at 931 MHz at peak load. This equates to roughly 9.5% of the Jetson's total theoretical compute potential based on a maximum of 100% utilization at 2201 MHz. This leaves over 90% of the CPU capacity available for other independent robotic workloads.

The implications of this lightweight architecture for mobile robotic platforms such as Boston Dynamics Spot are profound. Spot utilizes a 605 Wh battery, yielding an average uptime of roughly 90 minutes, which translates to an average operational power draw of ~ 403 W [4]. By appending the gesture pipeline's 2.38 W overhead, the total robot power draw increases to ~ 405.38 W. Consequently, running the

complex real-time AI gesture control system reduces the robot's total uptime by a mere 31 seconds, representing a 0.59% reduction.

5.4 User Study Results

The survey data, comprising ordinal Likert scales and Borda rankings, underwent a comprehensive three-stage non-parametric statistical evaluation. Because individual Likert scale responses produce strictly ordinal data where the distance between points cannot be mathematically assumed to be equal, standard parametric tests are inappropriate. Consequently, robust non-parametric methods were employed [26]. First, a one-sample Wilcoxon signed-rank test against a neutral baseline rating was conducted for each metric. This initial test confirms that positive participant ratings are not merely the result of random variation but stem from a statistically significant underlying distribution leaning toward a positive consensus. Next, the non-parametric Friedman test was applied to determine whether statistically significant differences exist across the five distinct gesture types. Upon finding global significance, post-hoc analysis was executed using pairwise Wilcoxon signed-rank tests to identify exactly which gestures significantly outperformed the others. Evaluating the five gesture types against one another requires ten individual pairwise comparisons. At a standard significance level of 0.05, conducting ten separate tests introduces a compounded 50% probability of encountering a false positive result by chance alone. To strictly control this family-wise error rate, a Bonferroni correction was implemented. Consequently, the standard alpha threshold is divided by the ten comparisons, requiring an adjusted significance level of 0.005 to definitively declare a statistical difference between any two gesture types.

5.4.1 Core Gesture Mechanics Evaluation

The foundational objective of evaluating the core gesture mechanics was to determine whether a vision-based hand gesture control scheme could provide a consistently positive user experience. The results decisively validate this hypothesis. A one-sample Wilcoxon signed-rank test confirmed that every single gesture type received statistically significant positive ratings against the neutral baseline. This held true across all three primary evaluation metrics of precision, naturalness, and effort, as detailed in Figure 5.11, Figure 5.12, and Figure 5.13. The vast majority of these ratings achieved high significance levels well below the 0.01 threshold. Consequently, participants did not merely tolerate the gesture control system but actively perceived it as a precise, natural, and low-effort modality. This uniform positive consensus across a diverse array of distinct gesture topologies serves as a major argument for the viability and success of the developed vision-based interface. Following this confirmation of general efficacy, the Friedman test was applied, revealing highly significant variations in how participants rated the different gestures across precision, naturalness, and effort. These variations suggest that while all gestures are viable, certain interaction paradigms are inherently preferred.

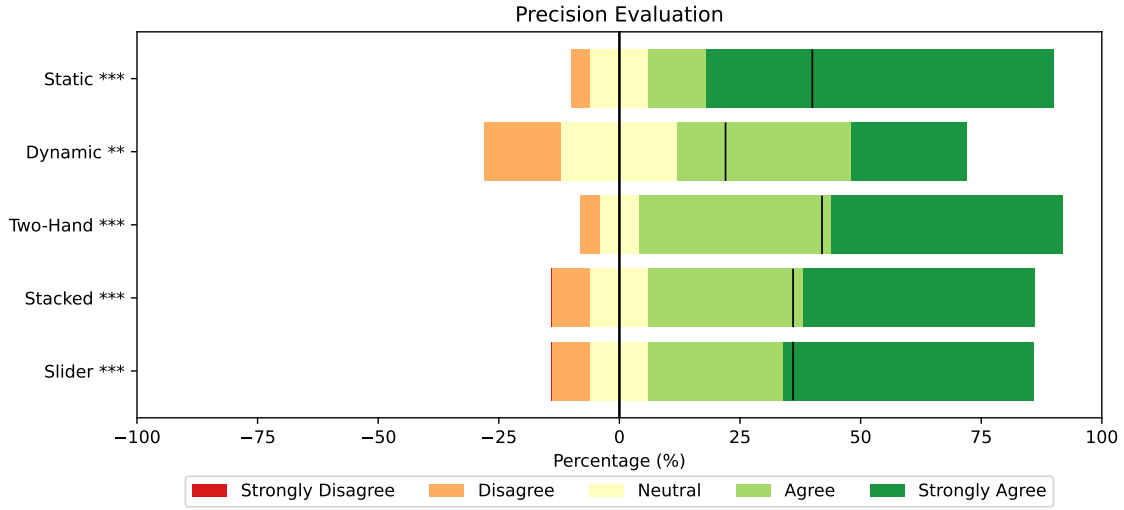


*Figure 5.11: Perceived precision across the five core gesture types. Asterisks denote significance against a neutral baseline * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$.*

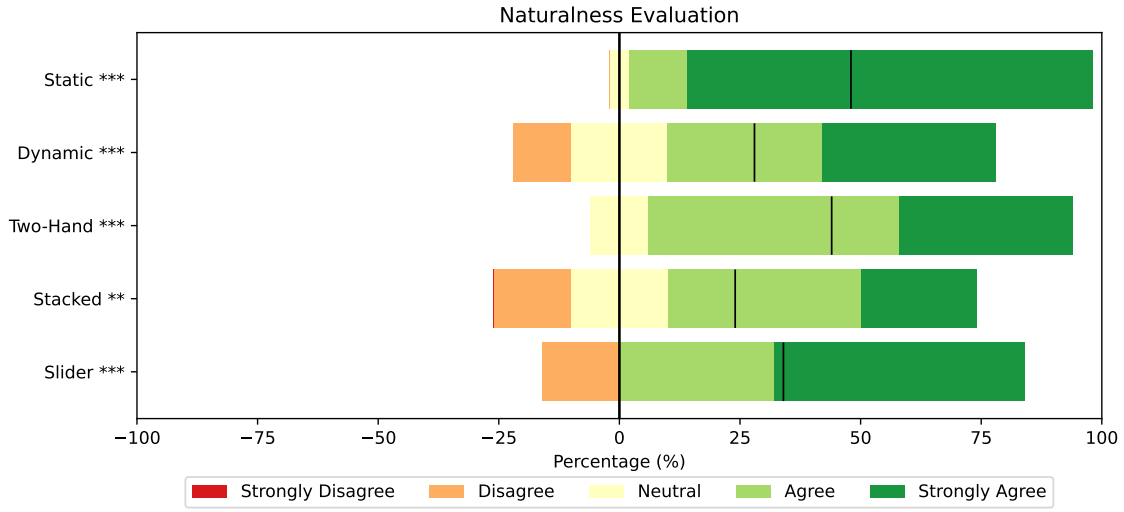


*Figure 5.12: Perceived naturalness across the five core gesture types. Asterisks denote significance against a neutral baseline * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$.*

The significant variations identified by the Friedman test were further analyzed through post-hoc pairwise analysis. The data reveals a definitive hierarchy favoring mechanical simplicity. While perceived precision exhibited no statistically significant pairwise differences after correction, the user experience metrics tell a compelling story. Static gestures significantly outperformed dynamic, stacked, and two-hand gestures in naturalness. Furthermore, static gestures proved significantly superior to every other tested gesture category in terms of required effort. This consistent preference unequivocally indicates that simpler single-pose interactions are strongly favored over complex multi-stage or temporally dependent movements. The superiority of static gestures is directly attributable to their minimal cognitive load,

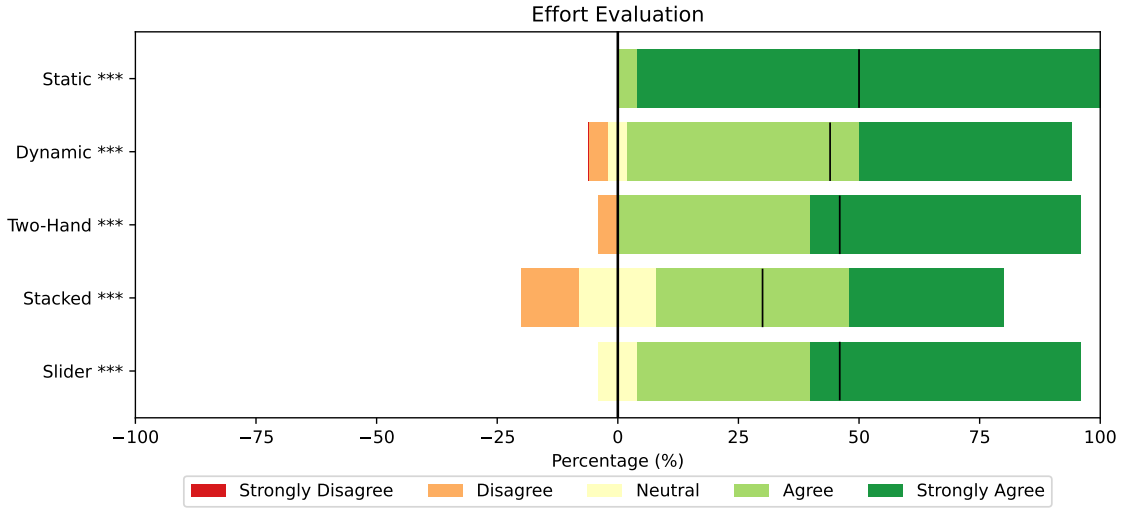


*Figure 5.13: Perceived physical and mental effort across the five core gesture types. Asterisks denote significance against a neutral baseline * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$.*

physical demand, and inherent error recovery mechanics. Holding a single continuous pose eliminates the necessity for precise timing, sequence memorization, or sustained dynamic coordination. Furthermore, if a static gesture is momentarily unrecognized by the inference pipeline, the user intuitively sustains the pose until detection succeeds. This creates a seamless, low-effort recovery loop. Conversely, a failed detection of a dynamic gesture forces the user to physically reset and repeat the entire motion sequence. This strict necessity to restart an action interrupts operational flow and significantly degrades the user experience, ultimately reinforcing the quantitative preference for static interactions.

5.4.2 System Mechanics & Tablet Comparison

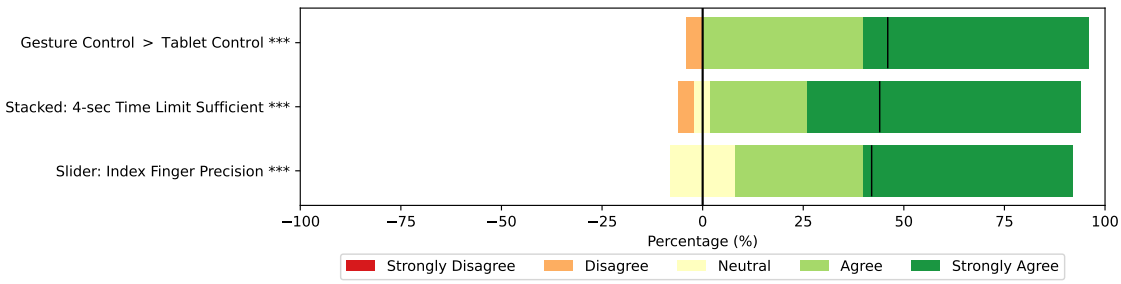


*Figure 5.14: User evaluation of specific system mechanics and overall comparison against the standard tablet controller. Asterisks denote significance against a neutral baseline * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$.*

Beyond evaluating generalized gesture categories, the survey assessed specific systemic implementations, the results of which are depicted in Figure 5.14. A one-sample Wilcoxon signed-rank test confirmed a statistically significant positive con-

sensus regarding two key design choices. First, it affirmed the sufficiency of the four-second execution window for stacked gestures. Second, it validated the perceived precision of utilizing the index finger, rather than the geometric center of all hand landmarks, as the spatial reference point for the slider mechanism. Crucially, the most pivotal finding of the study emerged when benchmarking the vision-based interface against the standard tablet controller during the simulated warehouse manipulation task. Participants demonstrated a highly significant agreement that the gesture control system offered a superior operational experience in the given situation. When asked to identify the specific factors driving this preference, as quantified in Figure 5.15, 80% of participants, amounting to 20 out of 25 individuals, cited the ability to maintain "free hands" during operation. Additionally, 56% of respondents valued the "natural interaction" paradigm, while 52% noted that the vision-based system was noticeably faster than using the physical tablet. This statistical consensus is further supported by qualitative free-text feedback. One user highlighted the immense practical advantage that "clean hands are not necessary," allowing seamless operation even when a worker's hands are soiled by machinery oil, grease, or physical dirt. Furthermore, another participant praised the elimination of the "searching time for the tablet," a delay commonly induced when workers forget where they placed the controller within a large operational area. A third noted that "requiring no extra device for operation" significantly streamlined the workflow. Ultimately, the synthesis of these pragmatic benefits culminated in a respondent describing the gesture-based interaction as feeling "Magical."

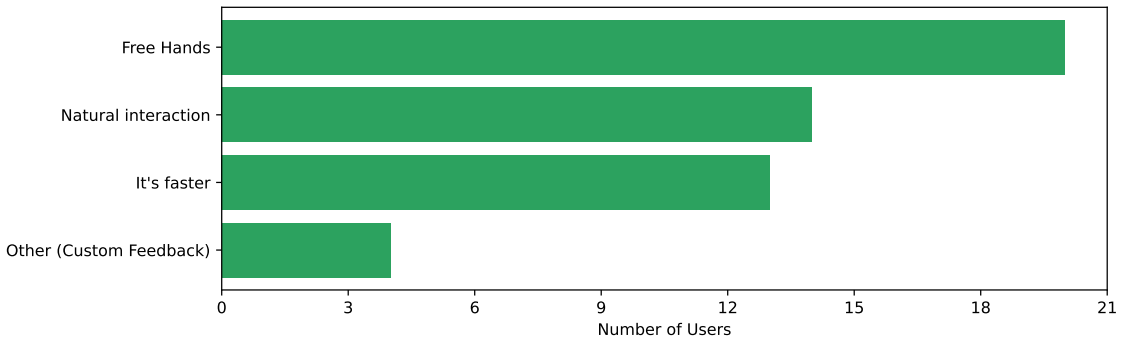


Figure 5.15: Frequency of preferred gesture control aspects cited by participants.

5.4.3 Alignment with Broader Academic Consensus

The highly favorable user perception of the gesture control interface documented in this study is strongly supported by larger-scale experiments within the field of industrial HRI. While the simulated warehouse manipulation task conducted in this thesis represents a focused, small-scale evaluation, its findings mirror the broader academic consensus. For instance, a comprehensive comparative analysis by Kristof et al. evaluated traditional teach pendants against wearable gesture control devices [28]. Despite the teach pendant offering superior objective precision, 85% of

their participants favored the gesture control interface, citing high overall satisfaction. This directly parallels the findings of this study, where users preferred gesture interactions for their naturalness and the ability to maintain free hands, even when standard tablet controllers were available.
Similarly, a recent study by Vanc et al. comparing kinesthetic, joystick, and gesture-based teaching modalities for non-expert users concluded that gesture-based teaching serves as a highly viable and intuitive alternative, particularly when physical interaction with the robot is constrained [60]. Their results emphasize that while gesture interfaces may sometimes lack the absolute millimeter-level reliability of traditional joysticks or teach pendants, the significantly lowered cognitive load and the elimination of complex hardware make them a highly preferred modality for operators. Ultimately, by cross-referencing the localized results of this thesis with these extensive benchmark studies, it becomes evident that the subjective preference for ergonomic, hands-free gesture control is not an isolated phenomenon, but rather a fundamental human-centric shift in the future of robotics interaction.

5.4.4 Overall Gesture Preference

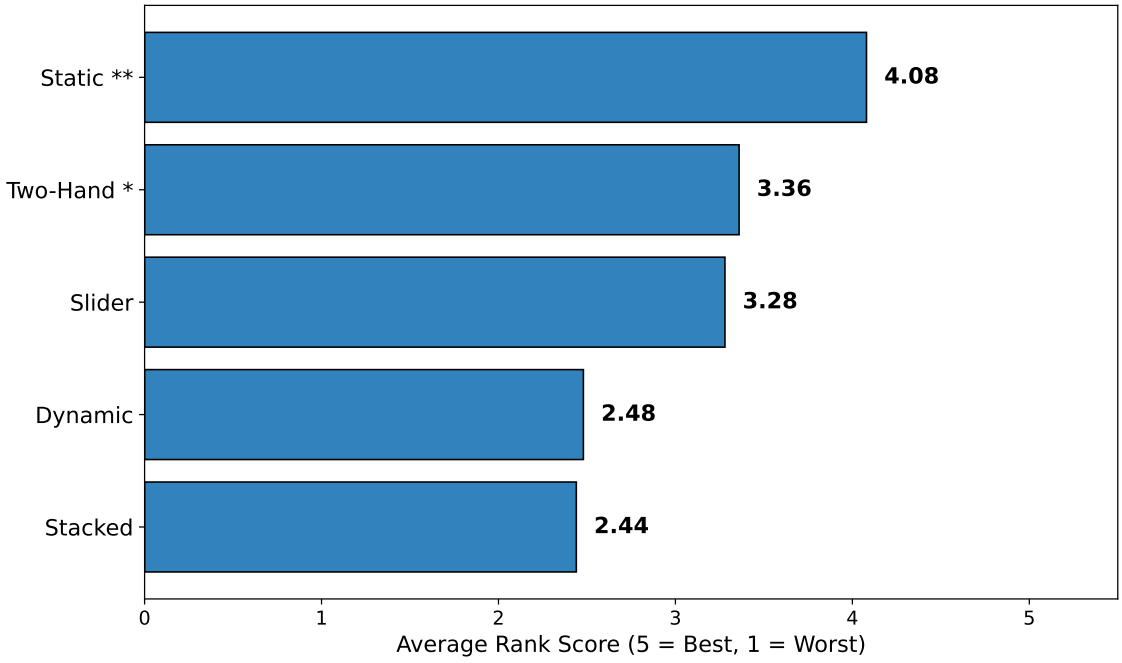


*Figure 5.16: Overall user preference rankings of the five gesture types, calculated using the Borda Count average rank score method. Asterisks denote significance against a neutral baseline * $p < 0.05$, ** $p < 0.01$.*

To visualize the ordinal ranking data collected from the survey, a Borda Count average rank score method was applied, yielding the hierarchical distribution presented in Figure 5.16. The statistical significance of these rankings was subsequently evaluated using a Friedman test, followed by post-hoc pairwise Wilcoxon signed-rank

tests. The analysis revealed that static gestures emerged as the significantly preferred champion, outperforming both dynamic and stacked gestures. Furthermore, stacked gestures were found to be significantly less preferred than two-hand gestures.

5.5 Summary

The comprehensive results of the user study yielded exclusively positive feedback regarding the vision-based control system. The data definitively established that when evaluating gesture types against one another, users consistently prefer simpler interactions over temporal or sequentially complex gestures. Furthermore, the strong preference for the gesture system over the standard tablet controller is primarily driven by the practical ability to maintain free hands and the intuitive, natural interaction paradigm, reinforcing the data from Figure 5.15. Ultimately, these findings confirm that ergonomic freedom and mechanical simplicity are the most critical determinants of a successful human-robot interface.

6 Limitations

While the proposed hybrid architecture successfully enables robust real-time gesture control on edge hardware, the system is constrained by several fundamental limitations. These boundaries arise from deliberate architectural trade-offs, physical hardware constraints inherent to mobile robotics, and biases embedded within the chosen training datasets. Acknowledging these limitations is critical for defining the operational envelope of the deployed system and identifying necessary improvements for future iterations.

6.1 Algorithmic and Architectural Constraints

The decision to decouple the gesture recognition pipeline into a distinct spatial extractor and a subsequent temporal classifier introduces a strict architectural limitation regarding feature learning. By separating these two processes, the system forfeits the ability to perform end-to-end spatio-temporal learning. If the MediaPipe spatial extractor fails to identify a hand due to severe motion blur or environmental occlusion, the GRU is fed uninformative zero-vectors. The temporal classifier is therefore entirely blind to the raw image data. A fully integrated end-to-end architecture, such as a 3D-CNN processing raw video frames, might learn to extract temporal motion patterns directly from blurry pixel textures even when explicit spatial landmarks cannot be formed. The decoupled approach prioritizes computational efficiency over this robust end-to-end feature abstraction.

Furthermore, the system imposes rigid temporal boundaries on human motion. The developed architecture relies on a 40-frame sequence buffer for dynamic gesture classification, representing approximately 3 seconds of motion. Additionally, to prevent spurious activations, all recognized static and dynamic gestures are passed through a secondary `GestureFilter`. This filter applies a 7-frame stability window, requiring 70% predictive consistency before a command is dispatched to the robot. While effective at rejecting noise, these discrete computational windows conflict with continuous HRI. Because the temporal classifier was trained on the Jester dataset, a collection consisting entirely of isolated, single-gesture video clips, it expects a clear, distinct motion pattern. If an operator performs rapid, sequential commands without briefly pausing in a neutral rest state, the 40-frame buffer inadvertently captures overlapping kinematic motions from multiple distinct gestures, severely degrading classification accuracy. Furthermore, while short gestures are easily accommodated

via zero-padding, gestures performed exceptionally slowly exceed the 3-second limit and are physically truncated. However, this truncation is rarely encountered during practical teleoperation, as the defined gesture classes natively possess short kinematic execution times that comfortably fit within this temporal window.

6.2 Hardware and Embodiment Constraints

A primary constraint governing the deployed pipeline is a framerate ceiling dictated by both the training data and the robotic edge hardware. The Jester dataset utilized for training the temporal classifier is natively standardized at 12 FPS. This low framerate establishes that 12 FPS is inherently sufficient to capture the temporal dynamics of human gestures while maintaining a highly favorable trade-off between computational efficiency and classification accuracy. While it is theoretically possible to use temporal interpolation to artificially augment the dataset to 30 FPS, this effort is fundamentally unnecessary for mobile robotics. Lower framerates are strictly more resource-efficient, which is critical for battery-powered autonomous systems. Concurrently, the NVIDIA Jetson AGX Orin limits standard USB Video Class webcam frame retrieval at approximately 14 FPS [47] without a custom hardware-accelerated GStreamer pipeline. Rather than viewing this I/O bottleneck as a detriment, it aligns perfectly with the operational realities of the target platform. The internal perception cameras and standard Robot Operating System data pipelines on the Boston Dynamics Spot are typically throttled to roughly 10 Hz to conserve network bandwidth and compute. Aligning the external gesture recognition pipeline to this lower framerate seamlessly matches the native constraints of modern robotic operating systems, maximizing energy efficiency without sacrificing operational accuracy.

A secondary hardware limitation is the reliance on synthetic depth. The system processes a standard 2D RGB video feed. While spatial augmentations and coordinate normalization successfully make the model scale-invariant, the Z-axis depth generated by MediaPipe remains purely synthetic and relative. Utilizing true 3D sensors, such as LiDAR, would theoretically provide absolute 3D hand topology. However, this is practically impossible with current standard robotic navigation hardware. The official Boston Dynamics Spot Enhanced Autonomy Payload utilizes a Velodyne VLP-16 LiDAR with a 2.0° vertical angular resolution. At a standard teleoperation distance of two meters, the vertical gap between laser beams is roughly seven centimeters. An average human hand would only register two or three data points, making fine-grained topological extraction mathematically impossible. Therefore, relying on 2D RGB cameras with synthetic depth estimation is a strict physical necessity for fine motor recognition at a distance.

Beyond internal processing limits, the physical embodiment of the robot introduces significant ergonomic constraints. Spot's built-in stereo cameras face downward to detect obstacles and walking paths. Forcing an operator to interact with these native cameras would require them to crouch or bend over significantly, which is ergonomically impractical for prolonged teleoperation. Because the quadruped cannot

physically walk and tilt its body upward simultaneously without disrupting its locomotion kinematics, a hardware workaround was mandatory. This required mounting a secondary upward-facing camera on the payload rail, proving that native out-of-the-box teleoperation is physically limited by the robot's default morphological design. Nevertheless, this limitation is strictly tied to the quadrupedal morphology. The developed software stack could be seamlessly transferred to a humanoid robot, where integrated head-mounted cameras naturally reside at the correct ergonomic height.

Furthermore, operating in dark industrial environments introduces significant optical constraints. In low-light conditions, standard cameras automatically increase exposure times to gather sufficient light, which inevitably leads to motion blur during rapid dynamic gestures. Because the spatial extractor relies on sharp edge detection to identify finger topology, motion blur causes immediate tracking failures. This environmental limitation can be mitigated through two primary hardware adjustments. The first is utilizing external cameras with larger apertures to natively increase light gathering. The second is manually forcing a high shutter speed to freeze human motion. While a high shutter speed requires a proportional increase in ISO sensitivity, thereby injecting digital grain into the image, modern spatial extractors are highly resilient against ISO-induced noise. Therefore, prioritizing high ISO over long exposure times is a necessary and effective mitigation strategy for low-light deployment.

Finally, effective gesture control requires immediate visual confirmation that the robot understood the human command. However, Spot's native front light-emitting diodes (LEDs) are strictly reserved for critical motor control or leash status for safety reasons. The inability to safely hijack these localized LEDs for gesture feedback forces reliance on external hardware or web dashboards.

6.3 Dataset and Generalization Biases

The final category of limitations stems from the statistical biases present within the training data and the environmental discrepancies between the training domain and the target robotic domain. The reported 91.63% validation accuracy achieved by the GRU in Chapter 5 was calculated after systematically dropping video sequences with extreme tracking failures. Specifically, videos containing fewer than nine valid spatial landmarks were filtered out to isolate the temporal classification performance from spatial extraction failures. Consequently, this high accuracy metric is technically bound to ideal tracking conditions. It does not perfectly reflect real-world performance, where the spatial extractor may drop tracking frequently due to the aforementioned optical and physical constraints. The reported accuracy represents the theoretical maximum performance of the temporal classifier given noise-free input vectors, rather than the raw end-to-end performance of the entire pipeline.

Furthermore, while the decoupled architecture renders the temporal classifier invariant to environmental backgrounds, the system remains susceptible to morphological

domain shift regarding hand appearance. The MediaPipe spatial extractor is heavily optimized for the color and texture heuristics of bare human skin, mirroring the bare-handed Jester dataset. In industrial settings where operators wear thick, uniformly colored protective gloves, the spatial extractor struggles to identify individual finger joints.

7 Future Work

While the proposed hybrid gesture recognition architecture successfully achieves robust real-time control, several avenues for future research remain to further expand the operational capabilities of the system.

7.1 Continuous Streaming Architectures

The current implementation relies on a rigid temporal buffer, storing a fixed sequence of frames before executing classification. As discussed in Chapter 6, this discrete buffering forces artificial rest states and conflicts with the fluid cadence of continuous HRI. Future architectures should transition from a static-window temporal model to a true streaming GRU. A streaming architecture natively digests fluid, variable-speed motion without requiring predefined sequence lengths. Furthermore, streaming models are highly computationally efficient during inference, as they only require a single forward pass per incoming frame to update their internal hidden state, rather than recalculating the entire historical sequence. However, transitioning to a continuous streaming paradigm introduces significant complexities during the training phase. Researchers must determine how to structure the data ingestion, such as whether to optimize the network after every single frame or to process variable-length chunks. A critical challenge lies in managing the internal memory state between distinct actions. The training pipeline must dictate whether to concatenate independent gesture videos back-to-back to simulate a continuous stream, or to manually flush the GRU's hidden state between clips to prevent kinematic contamination. Furthermore, deploying a streaming GRU introduces a known trade-off regarding predictive accuracy. True streaming models are strictly unidirectional. They cannot utilize bidirectional processing to glean context from future frames. Therefore, future research must carefully balance the ergonomic benefits of continuous streaming against these training complexities and the inherent loss of bidirectional context.

7.2 True Bimanual Interaction

The current gesture control implementation relies on simultaneous dual-hand control. In this paradigm, both the left and right hands act as completely independent

triggers, managed in parallel through separate software threads and synchronized via basic timers. While effective for issuing concurrent commands, this approach fails to capture complex inter-hand contact dynamics. True bimanual interaction involves the hands working in concert, where the spatial relationship between the fingers of both hands dictates a single, highly dexterous composite command. To transition toward genuine bimanual control, future systems must be trained on large-scale interacting hand datasets, such as the InterHand2.6M dataset [44]. Training the spatial and temporal networks to understand intricate left-right hand occlusions and relative skeletal distances would allow the robot to interpret vastly more complex, human-like manipulations rather than relying on two isolated, parallel control streams.

7.3 Online Few-Shot Learning and Continual Adaptation

To further enhance operational flexibility, it would be highly advantageous to empower operators to define and train novel, user-specific gestures directly in the field without initiating a full, cloud-based retraining cycle. However, updating a deployed model on the edge using traditional linear classification heads inevitably leads to catastrophic forgetting, where the network unlearns the original gesture classes in favor of the new live demonstrations. To enable robust online few-shot learning, future architectures should transition toward a metric learning paradigm. By replacing the dense classification layer with a Prototypical Network [54], the GRU acts exclusively as a feature extractor. Classification is then determined via cosine similarity between the live embedding and dynamically stored class centroids in the latent space. This centroid comparison mechanism allows operators to perform a live ten-shot calibration to add new gesture classes instantly without degrading the existing model weights. To ensure these few live examples generalize effectively, geometric augmentations, such as spatial jitter, structural scaling, and temporal time-warping, must be applied directly to the extracted skeletal vectors on the edge prior to centroid calculation.

7.4 Resolving Morphological Domain Shift

As identified in Chapter 6, the system currently struggles with severe morphological domain shifts caused by thick, uniformly colored industrial Personal Protective Equipment. Because the temporal classifier is entirely reliant on the structural coordinates provided by the spatial extractor, tracking failures caused by thick gloves immediately degrade the downstream classification accuracy. The most direct avenue for future work to resolve this constraint involves fine-tuning the underlying MediaPipe spatial tracking backbone. Rather than collecting a substantial new video dataset to retrain the temporal GRU, future efforts should focus solely on collecting an annotated image dataset of operators wearing various industrial gloves.

Fine-tuning the spatial extractor to recognize joint locations on protective equipment will restore tracking robustness. Once the spatial coordinates are accurately extracted from gloved hands, the decoupled nature of the architecture ensures that the temporal classifier will interpret the motion correctly without any further retraining.

7.5 Hardware Feedback and Modality Evaluation

The deployment of the gesture recognition pipeline on the Boston Dynamics Spot revealed critical practical constraints regarding HRI in headless environments. As discussed in Chapter 6, the inability to utilize the native front LEDs for gesture feedback leaves the operator without immediate visual confirmation. Future iterations must prioritize the integration of external, payload-mounted LED hardware to provide direct system status and classification confidence back to the operator. Furthermore, while gesture control provides a silent and highly expressive method of teleoperation, its objective efficacy must be formally benchmarked against other modalities. Future work should evaluate the gesture-control pipeline against a pure audio or voice-control baseline in different industrial settings. While a combination of both vision and audio would theoretically yield the most robust multimodal system, isolating and comparing these modalities under extreme acoustic and optical noise is necessary to quantify the precise operational benefits of purely visual interaction.

8 Conclusion

This thesis addressed the cognitive and physical burdens associated with traditional robotic teleoperation by developing a robust, hands-free gesture control interface for industrial environments. By implementing a decoupled, hybrid architecture, the system successfully bridged the gap between complex spatial extraction and efficient temporal classification. Utilizing the Google MediaPipe framework, highly accurate three-dimensional hand landmarks were extracted in real time. These spatial coordinates were subsequently processed by a computationally lightweight GRU, ensuring rapid temporal sequence classification without overloading the edge hardware. Deployed on the NVIDIA Jetson AGX Orin and integrated with the Boston Dynamics Spot quadruped via the Robot Operating System, the developed pipeline achieved reliable, low-latency teleoperation, effectively proving the technical viability of deploying advanced vision-based control paradigms directly on mobile robotic platforms.

Beyond its architectural success, the empirical validation of the system yielded compelling insights into HRI preferences. Quantitative benchmarking confirmed high classification accuracy on the large-scale Jester dataset alongside sub-millisecond inference times, proving the computational efficiency of the recurrent network. However, the most critical findings emerged from the comprehensive two-part user study. Crucially, the system received statistically significant positive ratings across all evaluation metrics, confirming that users actively perceived the vision-based interface as precise, natural, and low-effort. When comparing fundamental interaction types, the results definitively established that operators strongly prefer mechanically straightforward, static gestures over temporally complex or multi-stage movements. Static interactions fundamentally minimized cognitive load and allowed for effortless, intuitive error recovery without forcing the user to restart entire action sequences. Furthermore, during the simulated warehouse manipulation task, participants favored the vision-based gesture interface over standard physical tablet controllers. The ability to maintain completely free hands during operation emerged as the decisive pragmatic advantage, confirming that the elimination of physical hardware constraints significantly enhances the subjective user experience.

While the deployed system successfully validated the core hypothesis of this research, several environmental and hardware constraints remain. Severe morphological domain shifts caused by industrial protective equipment, alongside the rigid temporal boundaries of fixed-buffer architectures, present ongoing challenges for real-world

deployment. However, the proposed future integration of few-shot metric learning, true continuous streaming models, and bimanual contact dynamics offer clear, mathematically sound pathways to resolve these limitations. Ultimately, by cross-referencing the localized results of this thesis with broader academic consensus, it becomes evident that the subjective preference for ergonomic, hands-free gesture control is not an isolated phenomenon. This work demonstrates that despite current technological constraints, vision-based spatial-temporal gesture recognition represents a fundamental, human-centric shift in the future of robotics interaction, laying a critical foundation for highly intuitive and scalable industrial teleoperation.

List of Figures

3.1 End-to-End Gesture Control Architecture 15
3.2 Operator Hand Distance Distribution 18
3.3 Spatial Hand Distribution Map 19
3.4 Dataset Handedness Distribution 19
3.5 Temporal Classification Architecture 22

4.1 Spot Robot Hardware Configuration 31
4.2 Multi-Threading Pipeline Architecture 36
4.3 Web UI Dashboard 38

5.1 Hand Detection Rate by Frame Position 44
5.2 Per-Video Detection Rate Distribution 47
5.3 Detection Distribution by Gesture Class 48
5.4 Consecutive Missing-Frame Runs 49
5.5 Detection Rate vs. Model Accuracy 50
5.6 Representative 8-Class Confusion Matrix 51
5.7 Model Accuracy vs. Computational Cost 52
5.8 Impact of Landmark Interpolation on Accuracy 54
5.9 System and CPU power draw profiling 58
5.10 CPU utilization and clock speed profiling 58
5.11 Perceived Precision Evaluation 60
5.12 Perceived Naturalness Evaluation 60
5.13 Perceived Effort Evaluation 61
5.14 System Mechanics Evaluation 61
5.15 Preferred Gesture Control Aspects 62
5.16 Overall Gesture Type Rankings 63

A.1 GRU Confusion Matrix 86

List of Tables

3.1 Impact of Dataset Volume on 3D-CNN Accuracy 13

5.1 Manual Discordant Frame Evaluation 44
5.2 State-of-the-Art Architecture Comparison 55
5.3 Inference latency benchmark of the GRU model across different runtime frameworks on the Jetson AGX Orin. 56

Bibliography

[1] Ahmad, M. I., Bernotat, J., Lohan, K., and Eyssel, F. Trust and cognitive load during Human-Robot interaction. *arXiv preprint arXiv:1909.05160* (2019).

[2] Arnab, A., Dehghani, M., Heigold, G., Sun, C., Lučić, M., and Schmid, C. Vivit: A video vision transformer. In *Proceedings of the IEEE/CVF international conference on computer vision* (2021), pp. 6836–6846.

[3] Boston Dynamics. Spot network connectivity and security guidelines. `https://support.bostondynamics.com/`, 2024.

[4] Boston Dynamics. Spot: the agile mobile robot. `https://bostondynamics.com/products/spot/`, 2024. Accessed: 2026-08-15.

[5] Cao, C., Zhang, Y., Zhang, C., and Lu, H. Egocentric gesture recognition using recurrent 3d cnns. In *2017 IEEE International Conference on Computer Vision Workshops (ICCVW)* (2017), IEEE, pp. 1273–1280.

[6] Carreira, J., and Zisserman, A. Quo vadis, action recognition? a new model and the kinetics dataset. In *proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (2017), pp. 6299–6308.

[7] Chakraborty, S., Sarkar, S., Paul, P., Bhattacharjee, S., and Chakraborty, A. Sign language recognition using landmark detection, GRU and LSTM.

[8] Che, Z., Purushotham, S., Cho, K., Sontag, D., and Liu, Y. Recurrent neural networks for multivariate time series with missing values. *Scientific reports 8*, 1 (2018), 6085.

[9] Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078* (2014).

[10] Choi, J., Gao, C., Messou, J. C. E., and Huang, J.-B. Why Can't I Dance in the Mall? learning to mitigate scene bias in action recognition. In *Advances in Neural Information Processing Systems* (2019), vol. 32.

[11] Chung, J., Gulcehre, C., Cho, K., and Bengio, Y. Empirical evaluation of gated recurrent neural networks on sequence modeling. In *NIPS 2014 Workshop on Deep Learning and Representation Learning* (2014).

[12] Chyung, S. Y., Roberts, K., Swanson, I., and Hankinson, A. Evidence-based survey design: The use of a midpoint on the Likert scale. *Performance Improvement 56*, 10 (2017), 15–23.

[13] Dodge, S., and Karam, L. Understanding how image quality affects deep neural networks. In *2016 Eighth International Conference on Quality of Multimedia Experience (QoMEX)* (2016), IEEE, pp. 1–6.

[14] Duan, H., Zhao, Y., Chen, K., Lin, D., and Dai, B. Revisiting skeleton-based action recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2022), pp. 2969–2978.

[15] Fankhauser, P., Blösch, M., Rodriguez, D., Kaestner, R., Hutter, M., and Siegwart, R. Kinect v2 for mobile robot navigation: Evaluation and modeling. In *2015 International Conference on Advanced Robotics (ICAR)* (2015), IEEE, pp. 388–394.

[16] Fayyaz, M., Bahrami, E., Diba, A., Noroozi, M., Adeli, E., Van Gool, L., and Gall, J. 3D CNNs with adaptive temporal feature resolutions. *arXiv preprint arXiv:2011.08652* (2020).

[17] Gong, Y., Liu, G., Xue, Y., Li, R., and Meng, L. A survey on dataset quality in machine learning. *Information and Software Technology 162* (2023), 107268.

[18] Google AI Edge. MediaPipe gesture recognizer model card, 2023. Accessed: 2026-07-13.

[19] Google for Developers. MediaPipe framework GPU support, 2024. Accessed: 2026-07-25.

[20] Google Research. MediaPipe hands (Lite/Full) model card. Model card, Google, 2021.

[21] Han, F., Reily, B., Hoff, W., and Zhang, H. Space-time representation of people based on 3D skeletal data: A review. *Computer Vision and Image Understanding 158* (2017), 85–105.

[22] Hara, K., Kataoka, H., and Satoh, Y. Can spatiotemporal 3d cnns retrace the history of 2d cnns and imagenet? In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition* (2018), pp. 6546–6555.

[23] Hendrycks, D., and Dietterich, T. Benchmarking neural network robustness to common corruptions and perturbations. In *International Conference on Learning Representations* (2019).

[24] Hinton, G., Vinyals, O., and Dean, J. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531* (2015).

[25] Hochreiter, S., and Schmidhuber, J. Long short-term memory. *Neural computation 9*, 8 (1997), 1735–1780.

[26] Jamieson, S. Likert scales: how to (ab)use them. *Medical Education 38*, 12 (2004), 1217–1218.

[27] Kiranyaz, S., Avci, O., Abdeljaber, O., Ince, T., Gabbouj, M., and Inman, D. J. 1D convolutional neural networks and applications: A survey. *Mechanical systems and signal processing 151* (2021), 107398.

[28] Kristof, R., Ciupe, V., Lovasz, E.-C., and Ismael, G. A comprehensive analysis of Human–Machine interaction: Teaching pendant vs. gesture control in industrial robotics. *Actuators 15*, 4 (2026), 210.

[29] Likert, R. A technique for the measurement of attitudes. *Archives of psychology* (1932).

[30] Lin, J., Gan, C., and Han, S. TSM: temporal shift module for efficient video understanding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (2019), pp. 7083–7093.

[31] Lipton, J. I., Fay, A. J., and Rus, D. Baxter's homunculus: Virtual reality spaces for teleoperation in manufacturing. In *IEEE International Conference on Robotics and Automation (ICRA)* (2018), pp. 225–232.

[32] Liu, L., and Shao, L. Learning discriminative representations from RGB-D video data. In *Twenty-Third International Joint Conference on Artificial Intelligence* (2013).

[33] Liu, Y., Yang, J., Perera, M., Ji, P., Kim, D., Xu, M., Wang, T., Anwar, S., Gedeon, T., Wang, L., and Qin, Z. Representation-Centric survey of skeletal action recognition and the ANUBIS benchmark. *IEEE Transactions on Pattern Analysis and Machine Intelligence (arXiv preprint arXiv:2205.02071)* (2024).

[34] Liu, Z., Ning, J., Cao, Y., Wei, Y., Zhang, Z., Lin, S., and Hu, H. Video swin transformer. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (2022), pp. 3202–3211.

[35] Loquercio, A., Maqueda, A. I., Del-Blanco, C. R., and Scaramuzza, D. Dronet: learning to fly by driving. *IEEE Robotics and Automation Letters 3*, 2 (2018), 1088–1095.

[36] Loshchilov, I., and Hutter, F. SGDR: stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)* (2017).

[37] Loshchilov, I., and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)* (2019).

[38] Loshchilov, I., and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)* (2019).

[39] Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., MacGlashan, C.-L., Bogo, F., et al. Mediapipe: A framework for building perception pipelines. In *CVPR Workshops* (2019).

[40] Mancini, M., Costante, G., Valigi, P., Ciarfuglia, T. A., Delmerico, J., and Scaramuzza, D. Towards domain independence for Learning-Based monocular depth estimation. *IEEE Robotics and Automation Letters 2*, 4 (2017), 1883–1890.

[41] Materzynska, J., Berger, G., Bax, I., and Memisevic, R. The Jester dataset: A Large-Scale video dataset of human gestures. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops* (2019).

[42] Micikevicius, P., Narang, S., Alben, J., Diamos, G. F., Elsen, E., García, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., and Wu, H. Mixed precision training. In *International Conference on Learning Representations (ICLR)* (2018).

[43] Molchanov, P., Yang, X., Gupta, S., Kim, K., Tyree, S., and Kautz, J. Online detection and classification of dynamic hand gestures with recurrent 3d convolutional neural network. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (2016), pp. 4207–4215.

[44] Moon, G., Yu, S.-I., Wen, H., Shiratori, T., and Lee, K. M. InterHand2.6M: a dataset and baseline for 3D interacting hand pose estimation from a single RGB image. In *European Conference on Computer Vision (ECCV)* (2020), Springer, pp. 548–564.

[45] Mukherjee, S., and Cha, S. GPU-Accelerated optimization of Transformer-Based neural networks for Real-Time inference. *arXiv preprint arXiv:2603.28708* (2026).

[46] NVIDIA. Jetson AGX Orin series supported power modes. `https://docs.nvidia.com/jetson/archives/r35.4.1/DeveloperGuide/text/SD/PlatformPowerAndPerformance/JetsonOrinNXSeriesAndJetsonAgxOrinSeries.html`, 2024. Accessed: 2026-08-15.

[47] NVIDIA Corporation. *NVIDIA Jetson Linux Developer Guide: Accelerated GStreamer*, 2023. Hardware decoding requirements for USB UVC cameras on Jetson architecture.

[48] Papadatou-Pastou, M., Ntolka, E., Schmitz, J., Martin, M., Munafò, M. R., Ocklenburg, S., and Paracchini, S. Human handedness: A meta-analysis. *Psychological bulletin 146*, 6 (2020), 481.

[49] Peng, X., et al. Lightweight visual dynamic gesture recognition system based on CNN-LSTM-DSA. *Sensors* (2024).

[50] Pooja, S., Prem Sai, N., Aravinth, A., Prithiviraj, R., and Manikandan, P. American sign language recognition using GRU and LSTM.

[51] Rehman, M. U., Ahmed, F., Khan, M. A., Tariq, U., Alfouzan, F. A., Alzahrani, N. M., and Ahmad, J. Dynamic hand gesture recognition using 3D-CNN and LSTM networks. *Computers, Materials & Continua 70*, 3 (2022), 4675–4690.

[52] Sabbarese, F., Magliulo, L., Carratù, P., and Romano, M. Hand gesture recognition using recurrent neural networks and synthetic data generation.

[53] Shotton, J., Fitzgibbon, A., Cook, M., Sharp, T., Finocchio, M., Moore, R., Kipman, A., and Blake, A. Real-time human pose recognition in parts from single depth images. In *CVPR 2011* (2011), Ieee, pp. 1297–1304.

[54] Snell, J., Swersky, K., and Zemel, R. Prototypical networks for few-shot learning. In *Advances in Neural Information Processing Systems* (2017), vol. 30.

[55] Song, Y., Demirdjian, D., and Davis, R. Continuous body and hand gesture recognition for natural human-computer interaction. In *2011 9th IEEE International Conference on Automatic Face and Gesture Recognition (FG)* (2011), IEEE, pp. 388–393.

[56] Srivastava, N., Hinton, G. E., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research 15*, 1 (2014), 1929–1958.

[57] Staniaszek, M., Flatscher, T., Rowell, J., Niu, H., Liu, W., You, Y., Skilton, R., Fallon, M., and Hawes, N. AutoInspect: towards Long-Term autonomous industrial inspection, 2024.

[58] Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., and Wojna, Z. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2016).

[59] Tan, M., and Le, Q. EfficientNet: rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning* (2019), PMLR, pp. 6105–6114.

[60] Vanc, P., Behrens, J. K., Hlaváč, V., and Stepanova, K. How should we teach robots? a comparison of kinesthetic, joystick, and Gesture-Based teaching. In *Cognition and Artificial Life (CAL/KUZ) 2026 conference* (2026).

[61] Vanc, P., Štěpánová, K., et al. How should we teach robots? a comparison of kinesthetic, joystick, and Gesture-Based teaching. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)* (2026).

[62] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. u., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems 30*. Curran Associates, Inc., 2017, pp. 5998–6008.

[63] Wagh, A., et al. Quantifying similarities between MediaPipe and a known standard for tracking 2D hand trajectories. *bioRxiv* (2023).

[64] Wagner, A. R., Rao, M. B., Xiao, X., and Pirk, S. Understanding fire through thermal radiation fields for mobile robots. *arXiv preprint arXiv:2602.19108* (2026).

[65] xutom. YOLOv8-Hand-Pose. `https://github.com/xutom/YOLOv8-Hand-Pose`, 2023.

[66] Yan, S., Xiong, Y., and Lin, D. Spatial temporal graph convolutional networks for skeleton-based action recognition. In *Thirty-second AAAI conference on artificial intelligence* (2018).

[67] Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., and Grundmann, M. Mediapipe hands: On-device real-time hand tracking. *arXiv preprint arXiv:2006.10214* (2020).

[68] Zhang, Y., Cao, C., Cheng, J., and Lu, H. EgoGesture: a new dataset and benchmark for egocentric hand gesture recognition. *IEEE Transactions on Multimedia 20*, 5 (2018), 1038–1050.

[69] Zhang, Y., Shi, L., Wu, Y., Cheng, K., Cheng, J., et al. Gesture recognition based on deep deformable 3D convolutional neural networks. *Pattern Recognition 107* (2020), 107416.

A
Appendix

A.1 Full Confusion Matrix

Due to the large number of target classes (27), the complete row-normalized confusion matrix is presented here to ensure legibility. As is common scientific practice for high-dimensional matrices, the graphic is centered and allowed to exceed standard margins to maximize clarity.

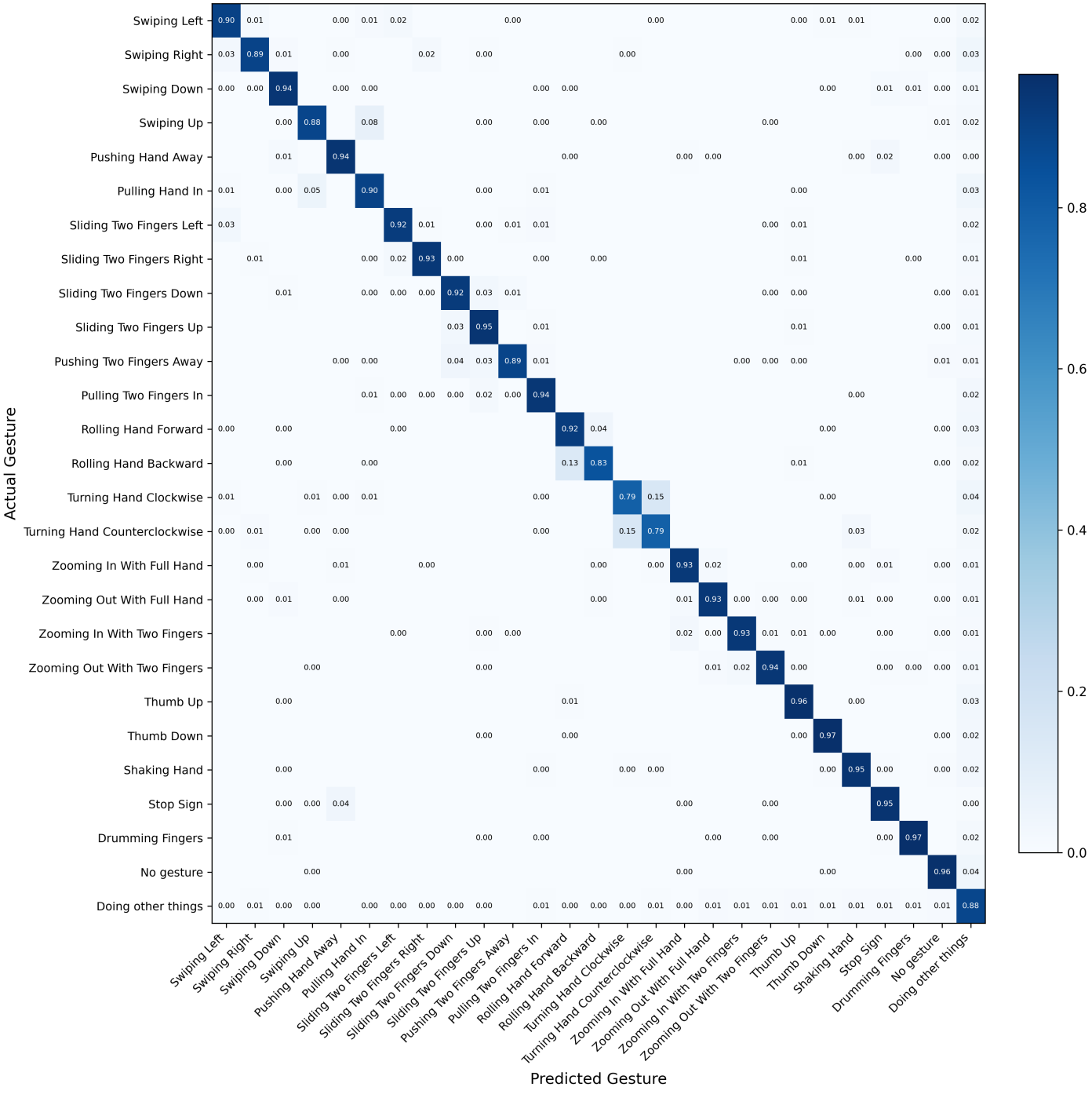


Figure A.1: Complete 27-class row-normalized confusion matrix of the GRU model evaluated on the validation set.